Westfälische Hochschule

University of Applied Sciences
Gelsenkirchen Bocholt Recklinghausen

Bachelorarbeit

Titel der Arbeit // Title of Thesis

Von Datenpunkten zu Graphen: Strukturanalyse medizinischer Daten im Kontext von Graphdatenbanken

Akademischer Abschlussgrad: Grad, Fachrichtung (Abkürzung) // Degree

Bachelor of Science (B.Sc.)

Autorenname, Geburtsort // Name, Place of Birth

Agasto Kuriyath Augustine, Manjapra

Studiengang // Course of Study

Praktische Informatik

Fachbereich // Department

Informatik und Kommunikation

Erstprüferin/Erstprüfer // First Examiner

Prof. Dr. Katja Zeume

Zweitprüferin/Zweitprüfer // Second Examiner

Prof. Dr. Laura Anderle

Abgabedatum // Date of Submission

03.12.2025

Eidesstattliche Versicherung

Kuriyath Augustine, Agasto
Name, Vorname // Name, First Name

Ich versichere hiermit an Eides statt, dass ich die vorliegende Abschlussarbeit mit dem Titel

Von Datenpunkten zu Graphen: Strukturanalyse medizinischer Daten im Kontext von Graphdatenbanken

selbstständig und ohne unzulässige fremde Hilfe erbracht habe. Ich habe keine anderen als die angegebenen Quellen und Hilfsmittel benutzt sowie wörtliche und sinngemäße Zitate kenntlich gemacht. Die Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Gelsenkirchen, 03.12.2025
Ort, Datum, Unterschrift // Place, Date, Signature

Einsatz von Textgenerierender KI

Im Rahmen der Erstellung dieser Bachelorarbeit wurden KI-gestützte Tools wie **ChatGPT** und **DeepL** verwendet. Diese Tools kamen insbesondere bei der grammatikalischen und stilistischen Überprüfung sowie der Optimierung von Textformulierungen zum Einsatz, um die sprachliche Qualität zu verbessern.

- **ChatGPT** wurde genutzt, um auf Grundlage meiner eigenen inhaltlichen Vorgaben Textentwürfe zu formulieren und diese sprachlich ansprechender zu gestalten. Zusätzlich habe ich einzelne Textabschnitte von ChatGPT umschreiben lassen, um die Verständlichkeit und Lesbarkeit zu verbessern. Die fachlichen Inhalte, Entscheidungen und endgültigen Formulierungen wurden von mir selbst erstellt, geprüft und verantwortet.
- **DeepL** Schreibassistent wurde verwendet, um zusätzliche Unterstützung bei der Rechtschreibprüfung und stilistischen Verfeinerung von Abschnitten zu leisten.

Die inhaltlichen Analysen, wissenschaftlichen Schlussfolgerungen und Implementierungen dieser Arbeit wurden vollständig eigenständig durchgeführt.
Die genannten KI-Tools dienten ausschließlich unterstützenden Zwecken und hatten keinen Einfluss auf die wissenschaftliche Integrität oder die Ergebnisse dieser Arbeit.

Gelsenkirchen, 03.12.2025

Ort, Datum, Unterschrift // Place, Date, Signature

Abstract

This thesis examines how medical and geographic data can be combined and analysed using a graph database. The goal of the work was to build a real graph model from public data sources and to evaluate how well this model supports different types of analyses. The database was created with Neo4j and includes information about cities, districts, postal codes, medical facilities, diseases, symptoms, medications, side effects, as well as statistical data such as birth statistics, death statistics, and Covid-19 statistics.

The study analyses several structural properties of the graph, such as strongly and weakly connected components, the graph diameter, and degree centrality. These analyses show that most parts of the graph are well connected, but some areas remain isolated because of missing or incomplete data. Practical examples, such as identifying possible diseases from a list of symptoms and comparing the density of pharmacies between two cities, demonstrate how the model can be used for real-world applications.

The results show that graph databases are well suited for linking complex medical and spatial information. Although some data limitations affected the completeness of the model, the overall structure works reliably. The thesis concludes that graph-based models offer valuable insights for healthcare analysis and provide a solid foundation for future extensions and research.

Danksagung

An dieser Stelle möchte ich meiner Mutter, Mary Augustine, meinen tiefsten Dank aussprechen. Ich weiß, wie schwer es für sie war, mich mit 18 Jahren für das Studium nach Deutschland ziehen zu lassen, besonders mit der großen finanziellen Belastung. Als alleinerziehende Mutter stand sie vor vielen Herausforderungen, doch sie hat meine Entscheidungen immer unterstützt und mir Kraft gegeben. Ohne ihre Stärke und ihre Opfer hätte ich diesen Weg nicht gehen können.

Mein herzlicher Dank gilt auch meinen Brüdern, Arun Augustine und Anson Augustine, die mich mein ganzes Leben lang begleitet haben. Schon in jungen Jahren haben sie neben ihrem eigenen Studium hart gearbeitet, um mich zu unterstützen und meinen Studienkredit zurückzuzahlen. Wir haben gemeinsam gekämpft, um unsere Mutter stolz zu machen – und heute reiche ich diese Arbeit ein, ein Traum, den wir viele Jahre geteilt haben. Schritt für Schritt werden unsere gemeinsamen Ziele nun Wirklichkeit.

Ebenso danke ich allen Familienmitgliedern – auf der Seite meines Vaters der Kuriyath-Familie und auf der Seite meiner Mutter der Kachappilly-Familie. Seit meiner Kindheit haben sie mich auf viele Arten unterstützt und mich in jeder Lebensphase ermutigt. Ihr Vertrauen und ihre Begleitung haben mich zu dem Menschen gemacht, der ich heute bin.

Mein aufrichtiger Dank gilt auch all meinen Freunden und ihren Familien, besonders jenen, die mir sehr nahestehen. Gemeinsam haben wir viele Herausforderungen gemeistert und uns auf diesem Weg gegenseitig unterstützt. Eure Hilfe hat mir in schwierigen Momenten Kraft gegeben.

Vielen Dank an alle Professorinnen und Professoren der Westfälischen Hochschule Gelsenkirchen, und ein besonderer Dank an Prof. Dr. Katja Zeume und Prof. Dr. Laura Anderle für ihre Unterstützung bei meiner Bachelorarbeit.

Inhaltsverzeichnis

1. Einleitung ... 1
1.1. Motivation ... 1
1.2. Zielsetzung & Forschungsfragen ... 1
1.3. Vorgehen und Aufbau der Arbeit ... 2
2. Grundlagen ... 3
2.1. Einführung in die Graphdatenbank ... 3
2.2. Grundlagen der Graphentheorie ... 3
2.3. Eigenschaften von Graph ... 6
2.3.1. Stark zusammenhängende Graphen ... 7
2.3.2. Schwach zusammenhängende Graphen ... 11
2.4. Neo4j und die Cypher-Abfragesprache ... 14
2.5. Analyseverfahren in Graphdatenbanken ... 18
Kürzester Pfad (Shortest-Path) ... 18
Längster kürzester Pfad (Graphdurchmesser) ... 19
Zentralitätsmaße ... 20
Gradezentralität ... 20
3. Datenbankaufbau und Herausforderungen ... 21
3.1. Ziel des Datenbankaufbaus ... 21
3.2. Aufbau der Entwicklungsumgebung ... 21
3.3. Datenbeschaffung und Probleme bei den Quellen ... 22
3.4. Datenaufbereitung und Transformation ... 23
3.5. Modellierungsprozess: Von erster Idee zum finalen Graphen ... 24
3.5.1. Erster Entwurf des Modells ... 24
3.5.2. Zweiter Entwurf des Modells ... 25
3.5.3. Finaler Entwurf des Schemas ... 26
4. Analyse der Graphstruktur und praktischer Anwendungsfälle ... 28
4.1. Analyse der stark und schwach zusammenhängenden Komponenten ... 28
4.1.1. Methodik ... 28
4.1.2. Ergebnisse ... 29
4.1.3. Interpretation der Ergebnisse ... 30
4.1.4. Diskussion ... 31
4.2. Durchmesser-Analyse ... 33
4.2.1. Methodik ... 34

4.2.2. Ergebnisse ... 35
4.2.3. Interpretation der Ergebnisse ... 36
4.2.4. Diskussion ... 37
4.3. Analyse der Grad-Zentralität ... 37
4.3.1. Methodik ... 38
4.3.2. Ergebnisse ... 39
4.3.3. Interpretation der Ergebnisse ... 41
4.3.4. Diskussion ... 41
4.4. Krankheitserkennung anhand von Symptomen ... 42
4.4.1. Methodik ... 42
4.4.2. Ergebnisse ... 44
4.4.3. Interpretation der Ergebnisse ... 44
4.4.4. Diskussion ... 45
4.5. Einwohnerzahl und Apothekenverteilung ... 46
4.5.1. Methodik ... 46
4.5.2. Ergebnisse ... 47
4.5.3. Interpretation der Ergebnisse ... 48
4.5.4. Diskussion ... 48
5. Diskussion ... 49
6. Schlussfolgerung ... 51
6.1. Fazit ... 51
6.2. Ausblick ... 51
Literaturverzeichnis ... 53

1. Einleitung

Daten spielen heute eine zentrale Rolle im Gesundheitswesen. Sie helfen dabei, medizinische Entwicklungen zu verstehen, Krankheitsverläufe zu analysieren und die Versorgung der Bevölkerung zu verbessern. Durch die kontinuierliche Erfassung großer Datenmengen entstehen neue Möglichkeiten, Muster früher zu erkennen und Risiken gezielt einzuschätzen.

1.1. Motivation

Die COVID-19-Pandemie hat eindrücklich gezeigt, wie wichtig gut strukturierte und verlässliche Daten für schnelle Entscheidungen sind. Eine zeitnahe Auswertung medizinischer und räumlicher Informationen kann helfen, Ausbrüche früh zu identifizieren und passende Maßnahmen einzuleiten.

Die wachsende Menge an medizinischen und räumlichen Daten stehen in enger Beziehung zueinander und müssen gemeinsam betrachtet werden, um ein vollständiges Bild zu erhalten. Klassische relationale Datenbanken stoßen dabei häufig an Grenzen, da sie komplexe Verbindungen nur indirekt abbilden. Graphdatenbanken bieten hingegen die Möglichkeit, solche Netzwerke direkt und übersichtlich darzustellen (Angles & Gutierrez, 2008).

Ein gut aufgebautes Graphmodell kann helfen, Zusammenhänge zwischen medizinischen Informationen sichtbar zu machen oder Versorgungsstrukturen besser zu analysieren. Dadurch lassen sich Gesundheitsrisiken früher erkennen und Entscheidungen gezielter treffen. Die Motivation dieser Arbeit besteht daher darin zu untersuchen, wie medizinische und räumliche Daten in einer Graphdatenbank strukturiert werden können und welches Potenzial solche Modelle für Analysen im Gesundheitswesen bieten

1.2. Zielsetzung & Forschungsfragen

. Ziel dieser Bachelorarbeit ist es,

- eine reale medizinisch-geografische Graphdatenbank aus öffentlichen Datenquellen aufzubauen,
- die strukturellen Eigenschaften dieses Graphen zu untersuchen,
- und zu prüfen, wie gut das Modell in praktischen Anwendungsfällen funktioniert.

Daraus ergeben sich die folgenden Forschungsfragen:

- Wie lassen sich medizinische und räumliche Daten effizient in einer Graphdatenbank modellieren?
- Welche Herausforderungen treten beim Aufbau einer solchen Datenbank auf?

- Welche praktischen Anwendungen im Gesundheitswesen können von einer Graphdatenbank profitieren?

1.3. Vorgehen und Aufbau der Arbeit

Die Arbeit ist so aufgebaut, dass sie Schritt für Schritt vom theoretischen Verständnis zur praktischen Anwendung führt. In Kapitel 2 werden die Grundlagen vorgestellt, die für das Verständnis von Graphdatenbanken und den späteren Analysen notwendig sind. Kapitel 3 beschreibt den Aufbau der Datenbank, die verwendeten Datenquellen und die Umsetzung des Modells in Neo4j. Darauf aufbauend untersucht Kapitel 4 sowohl die strukturellen Eigenschaften des Graphen als auch praktische Anwendungsfälle, die zeigen, wie das Modell eingesetzt werden kann. Kapitel 5 fasst die Analyse zusammen, bewertet sie und ordnet sie in den fachlichen Kontext ein. Abschließend formuliert Kapitel 6 das Fazit der Arbeit und bietet einen Ausblick auf mögliche Weiterentwicklungen.

2. Grundlagen

Dieses Kapitel vermittelt die theoretischen Grundlagen, die für die spätere praktische Umsetzung der Arbeit notwendig sind. Die folgenden Unterkapitel führen Schritt für Schritt in die wichtigsten Konzepte ein, die für das Verständnis des verwendeten Graphmodells und der durchgeführten Analysen erforderlich sind.

In den folgenden Unterkapiteln werden zunächst die Einführung in die Graphdatenbank (2.1), Grundlagen der Graphentheorie (Kapitel 2.2) sowie die grundlegenden Eigenschaften von Graphen (Kapitel 2.3) näher erläutert. Anschließend wird die Funktionsweise von Neo4j und die Cypher-Abfragesprache (Kapitel 2.4) vorgestellt. Abschließend werden Analyseverfahren in Graphdatenbanken (Kapitel 2.5) beschrieben, um ein Verständnis der Thematik zu vermitteln.

2.1. Einführung in die Graphdatenbank

Graphdatenbanken gehören zur Familie der NoSQL-Datenbanken und unterscheiden sich grundlegend von relationalen Datenbanksystemen, da sie keine Tabellen, sondern Knoten und Kanten als zentrale Strukturelemente verwenden. Eine Graphdatenbank ist ein Datenbanksystem, das Informationen in Form von Graphstrukturen speichert und verarbeitet. Sie bietet insbesondere bei der Verarbeitung verbundener Daten eine höhere Leistungsfähigkeit und Flexibilität, da sich neue Datenelemente und Beziehungen ohne aufwändige Schemaänderungen einfach in das bestehende Modell integrieren lassen. (Robinson et al., 2015). Um die Funktionsweise von Graphdatenbanken besser zu verstehen, ist es notwendig, zunächst die grundlegenden Konzepte der Graphentheorie zu betrachten.

2.2. Grundlagen der Graphentheorie

Ein **Graph** $G(V, E)$wird definiert durch eine endliche Knotenmenge V und eine Kantenmenge E, wobei jede Kante zwei unterschiedliche Knoten aus V verbindet (Teschl & Teschl, 2013, S. 416). Abbildung 1 zeigt ein Beispiel eines Graphen.

Abbildung 1

Graph mit drei Knoten und drei Kanten

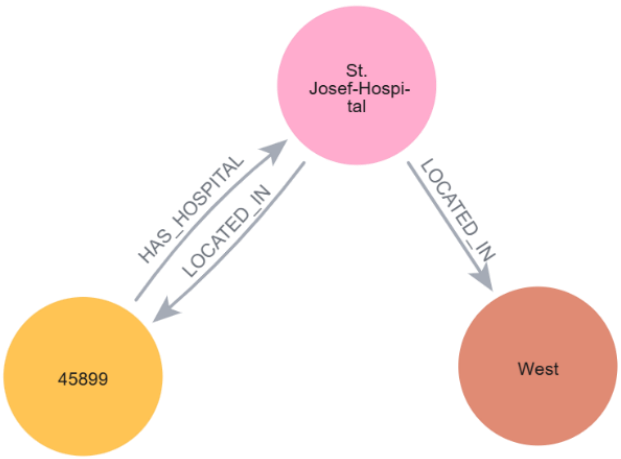


Abbildung 1 zeigt ein Graph mit drei Knoten (*St. Josef-Hospital*, *45899* und *West*) sowie zwei Kanten (*HAS_HOSPITAL* und *LOCATED_IN*), welche die Beziehungen zwischen den Knoten verdeutlichen.

Ein **Knoten** ist ein Element oder Objekt in einem Graphen, das eine Entität oder Einheit darstellt. In einer Graphdatenbank entspricht ein Knoten oft einer Dateninstanz beispielerweise einer Stadt, einem Krankenhaus, einer Person oder einer Krankheit. Eine **Kante** ist die Verbindung zwischen zwei Knoten. Sie beschreibt die Beziehung oder Interaktion zwischen diesen Entitäten. Knoten und Kanten können zudem Eigenschaften besitzen, die zusätzlichen Informationen, beispielsweise die Anzahl der Lebendgeborenen im Jahr 2025, beschreiben (Teschl & Teschl, 2013, S. 415–416).

- Zwei Knoten sind **adjazent** (Nachbarn), wenn sie durch eine Kante verbunden sind (Teschl & Teschl, 2013, S. 417). In Abbildung 1 sind die Knoten *45899* und das *St.-Josef-Hospital* adjazent, da zwischen ihnen eine direkte Kante besteht. Die Knoten *45899* und *West* sind nicht adjazent, da keine direkte Kante zwischen ihnen besteht.
- Der **Grad** eines Knotens *St. Josef Hospital*, bezeichnet als deg (St. Josef Hospital), gibt an, wie viele Kanten mit diesem Knoten verbunden sind. Es handelt sich also um die Anzahl der Kanten, die bei *St. Josef Hospital* beginnen oder enden. Der Grad kann in Eingangsgrad und Ausgangsgrad unterteilt werden (Teschl & Teschl, 2013, S. 417). Die folgenden Werte beziehen sich auf die in Abbildung 1 dargestellte Struktur:

- **Gesamtgrad:**
  deg (St. Josef Hospital) =3
- Der **Eingangsgrad** bezeichnet die Anzahl der Kanten, die auf einen Knoten zeigen, also die eingehenden Verbindungen. (Teschl & Teschl, 2013, S. 417)

  deg−(St. Josef Hospital) =1
- Der **Ausgangsgrad** gibt die Anzahl der Kanten an, die von einem Knoten ausgehen, also die ausgehenden Verbindungen (Teschl & Teschl, 2013, S. 417)

  deg+(St. Josef Hospital) =2

- **Isolierter Knoten:** Ein Knoten mit **Grad 0,** Isolierter Knoten haben keine Verbindung zu anderen Knoten (Teschl & Teschl, 2013, S. 417)
- Bei einem **gerichteten Graphen** besitzen die Kanten eine Richtung. Eine Kante von Medikamenten zu Nebenwirkung wird als (Medikamente, Nebenwirkung) notiert und stellt eine gerichtete Beziehung dar. Für die Traversierung bedeutet dies, dass Kanten nur in ihrer vorgegebenen Richtung durchlaufen werden können: Eine Bewegung vom Start- zum Zielknoten ist erlaubt, der umgekehrte Weg jedoch nicht, sofern keine entsprechende Rückkante existiert (Teschl & Teschl, 2013, S. 421).

Abbildung 2

Gerichteter Graph von Paracetamol 650 mg tablet und seinen Nebenwirkungen.

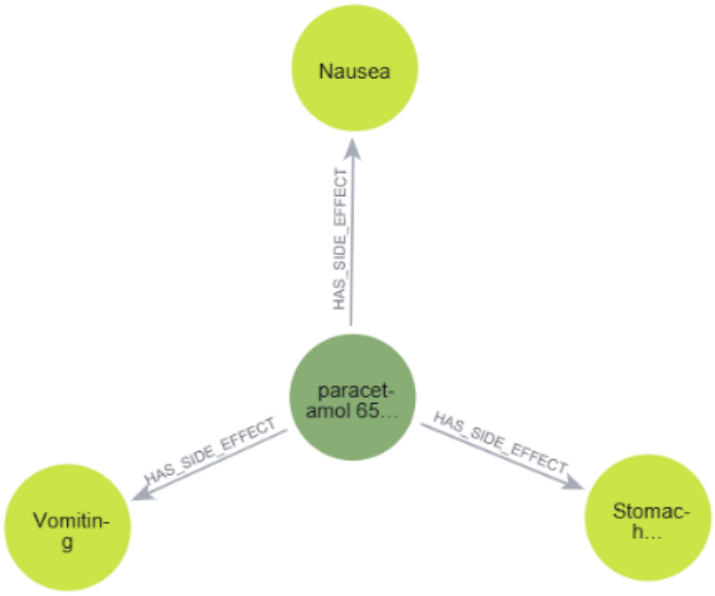
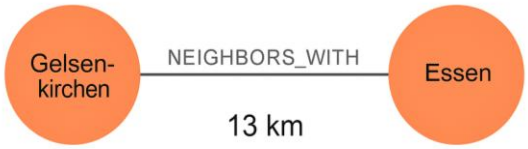


Abbildung 2 zeigt einen gerichteten Graphen, der den Knoten *Paracetamol 650 mg tablet* sowie die zugehörigen Nebenwirkungen über gerichtete Kanten darstellt.

- In einem **ungerichteten Graphen** haben die Kanten keine Richtung. Eine Verbindung zwischen den Knoten Gelsenkirchen und der Nachbarstadt Essen wird als [Gelsenkirchen, Essen] notiert und kann in beide Richtungen durchlaufen werden (Berghammer, 2024, S. 256–257).
  Abbildung 3 zeigt einen ungerichteten Graphen als Beispiel.

Abbildung 3

Ungerichteter Graph zwischen Gelsenkirchen und Essen.



- Ein **gewichteter Graph** ist ein Graph, bei dem den Kanten zusätzliche Werte zugeordnet werden. Diese Gewichte können Entfernungen, Kosten, Zeiten oder Kapazitäten darstellen (Teschl & Teschl, 2013, S. 449). In Abbildung 3 sieht man ein solches Beispiel: Das Gewicht der Kante zwischen Gelsenkirchen und Essen entspricht der Entfernung zwischen beiden Städten und beträgt 13 km.

2.3. Eigenschaften von Graph

Ein Graph G gilt als zusammenhängend, wenn es für jeden Knoten einen Weg zu jedem anderen Knoten gibt. Der größte solcher Teilgraph wird als Zusammenhangskomponente bezeichnet. Ein gerichteter Graph heißt zusammenhängend, wenn der zugehörige ungerichtete Graph diese Eigenschaft hat. Ein zusammenhängender Graph mit n Knoten benötigt mindestens $n-1$ Kanten. Besitzt ein Graph mehr als $\frac{(n-1)(n-2)}{2}$ Kanten, ist er notwendigerweise zusammenhängend (Teschl & Teschl, 2013, S. 426–428).

Die folgende Abbildung 4 zeigt die Krankheit *Anxiety* in der Datenbank mit insgesamt elf Knoten, bestehend aus dem Krankheitsknoten selbst sowie den zugehörigen Spezialität- und Medizine-Knoten. Zwischen diesen bestehen zehn Kanten, wodurch der Teilgraph zusammenhängend ist. Ein Graph mit $n = 11$ Knoten besitzt mindestens $n - 1 = 10$ Kanten, um zusammenhängend zu sein. Wenn er mehr als $\frac{(n-1)(n-2)}{2} = \frac{(11-1)(11-2)}{2} = 45$ Kanten besitzt, ist er notwendigerweise zusammenhängend.

Abbildung 4

Teilgraph der Krankheit Anxiety mit zugehörigen Specialty- und Medicine-Knoten

Im Spezialfall eines Graphen mit nur einem Knoten (n = 1) ergibt sich aus der Formel $\boldsymbol{n-1}$ eine minimale Kantenzahl von 0. Ein solcher Graph ist ohne Kanten zusammenhängend, da kein weiterer Knoten existiert. Er bildet damit eine Zusammenhangskomponente der Größe 1.

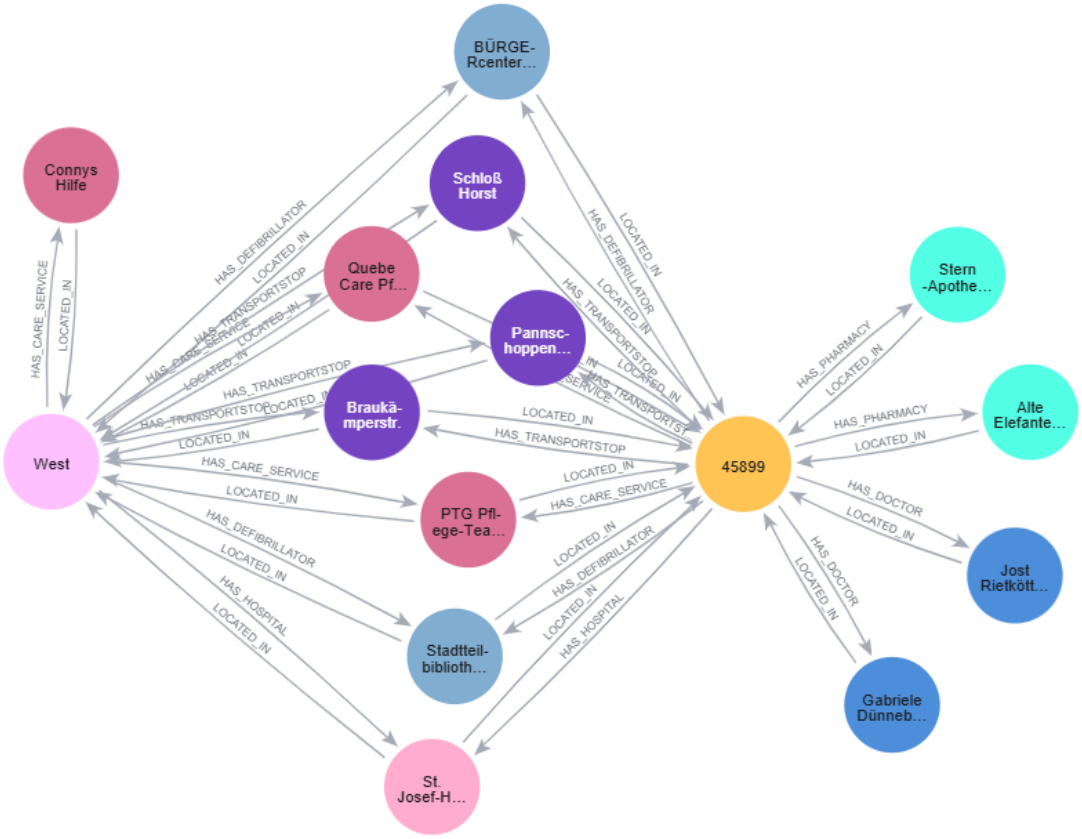
In den Unterkapiteln 2.3.1 und 2.3.2 werden die stark und schwach verbundenen Graphen sowie ihre Beziehung zu den Zusammenhangskomponenten näher erläutert

2.3.1. Stark zusammenhängende Graphen

Ein gerichteter Graph G= (V, E) heißt stark zusammenhängend, wenn für jedes Knotenpaar u,v∈V ein gerichteter Pfad von u nach v und umgekehrt existiert. Das bedeutet: Jeder Knoten ist unter Beachtung der Kantenrichtungen von jedem anderen aus erreichbar. (Georgiadis et al., 2020).

Abbildung 5

Darstellung eines stark zusammenhängenden Graph.



Eigenschaften stark verbundener Graphen

- **Bidirektionale Pfade:** Für jedes Knotenpaar u und v existieren gerichtete Pfade sowohl von u nach v als auch von v nach u. Diese Eigenschaft macht den Graphen stark verbunden.

  In Abbildung 5 wird die wechselseitige Erreichbarkeit am Knotenpaar West und 45899 verdeutlicht. Es existieren gerichtete Pfade in beide Richtungen *(West → St Josef-Hospital → 45899 und 45899 → Stadtteilbibliothek Horst, Vorraum → West).* Ein vergleichbares Verhalten lässt sich auch für alle andere Knotenpaare innerhalb des dargestellten Graphen beobachten.
- **Einzelne Komponente:** Ein stark verbundener Graph bildet genau eine stark zusammenhängende Komponente. Das Konzept der stark zusammenhängenden Komponenten wird im folgenden Unterkapitel näher erläutert.

- **Abgeschlossene Struktur:** Der Graph bildet eine geschlossene Struktur, da von jedem Startknoten aus allen anderen Knoten erreicht werden können und es möglich ist, anschließend wieder zum Ausgangspunkt zurückzukehren.

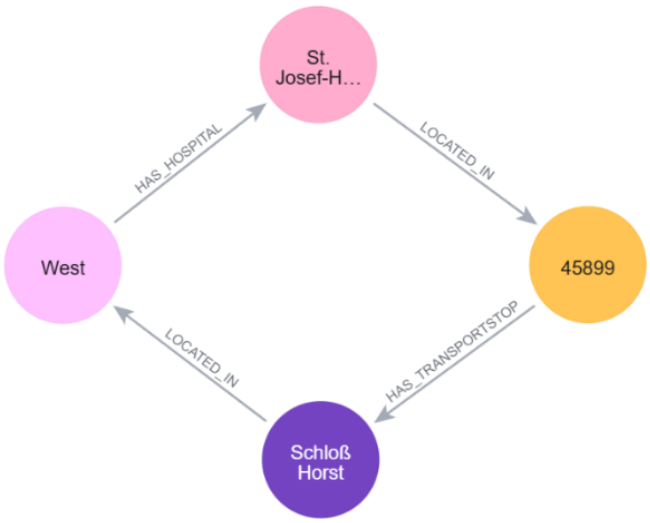
  In Abbildung 5 lässt sich dies daran erkennen, dass vom Knoten *West* aus der Pfad *West → St Josef-Hospital → 45899 → Stadtteilbibliothek Horst, Vorraum → West* durchlaufen werden kann. Ebenso existieren für die übrigen Knoten vergleichbare Rückwege, wodurch der Graph eine abgeschlossene und stark zusammenhängende Struktur aufweist.
- **Wiederkehrende Zyklen:** In einem stark zusammenhängenden Graphen existieren wiederkehrende Zyklen, die eine fortlaufende Traversierung zwischen Knoten in beliebiger Richtung erlauben und somit die innere Konnektivität sicherstellen. In den Abbildungen 6 und 7 sind Beispiele für Zyklen dargestellt, die aus dem in Abbildung 5 gezeigten stark zusammenhängende Graphen stammen.

Abbildung 6

Beispiel für einen Zyklus mit zwei Knoten.

Abbildung 7

Beispiel für einen Zyklus mit vier Knoten.

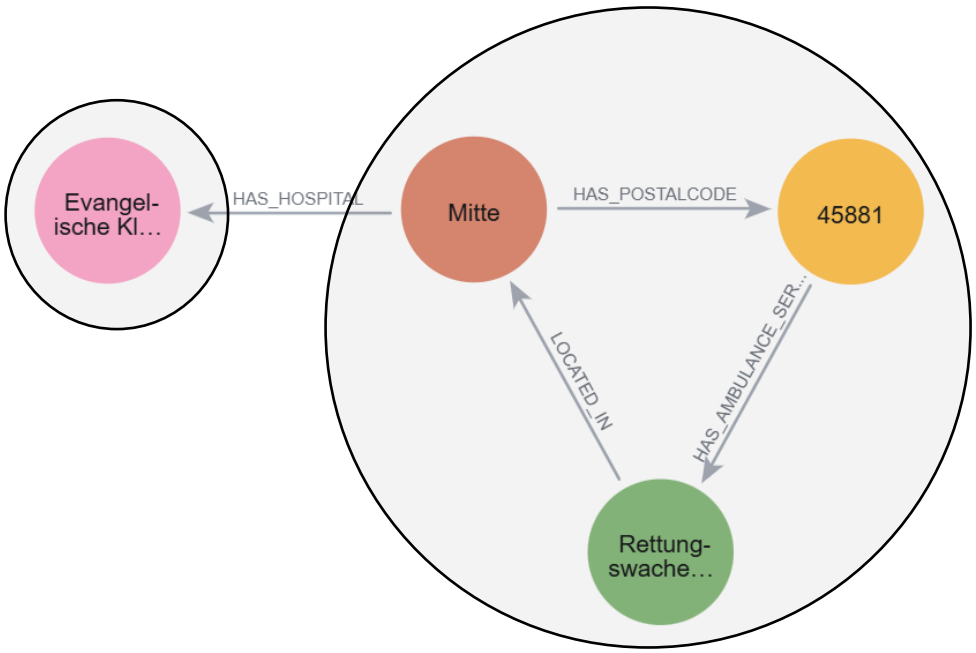


Stark zusammenhängende Komponenten

Eine stark zusammenhängende Komponente (engl. *strongly connected component*, SCC) ist ein maximaler Teilgraph eines gerichteten Graphen, in dem jeder Knoten von jedem anderen Knoten aus über gerichtete Pfade erreichbar ist. Ein isolierter Knoten bildet eine stark zusammenhängende Komponente (Heun, 2003, S. 161). Stark zusammenhängende Komponenten sind in vielen Bereichen hilfreich, weil sie zeigen, welche Knoten in einem Netzwerk besonders eng miteinander verbunden sind. In sozialen Netzwerken können solche Komponenten zum Beispiel Gruppen von Personen sichtbar machen, die ähnliche Interessen teilen oder häufig miteinander interagieren. Mithilfe von Graphalgorithmen lassen sich diese Gruppen zuverlässig identifizieren. Darauf aufbauend können anschließend passende Empfehlungen erstellt werden, etwa für Inhalte oder Werbeanzeigen, die für Personen mit ähnlichen Profilen relevant sind. Ein reales Graph kann mehr als eine stark zusammenhängende Komponente haben (Needham & Hodler, 2019, S. 120). Dies wird in Abbildung 8 veranschaulicht.

Abbildung 8

Graph mit zwei stark zusammenhängenden Komponenten



In Abbildung 8 sind zwei stark zusammenhängende Komponenten des Graphen dargestellt. Der Krankenhaus-Knoten *Evangelische Kliniken Gelsenkirchen GmbH* bildet die erste Komponente, während der Stadtbezirk *Mitte*, der Postleitzahlknoten *45881* und der Krankentransport *Rettungswache Schalke-Nord (31)* die zweite Komponente bilden.

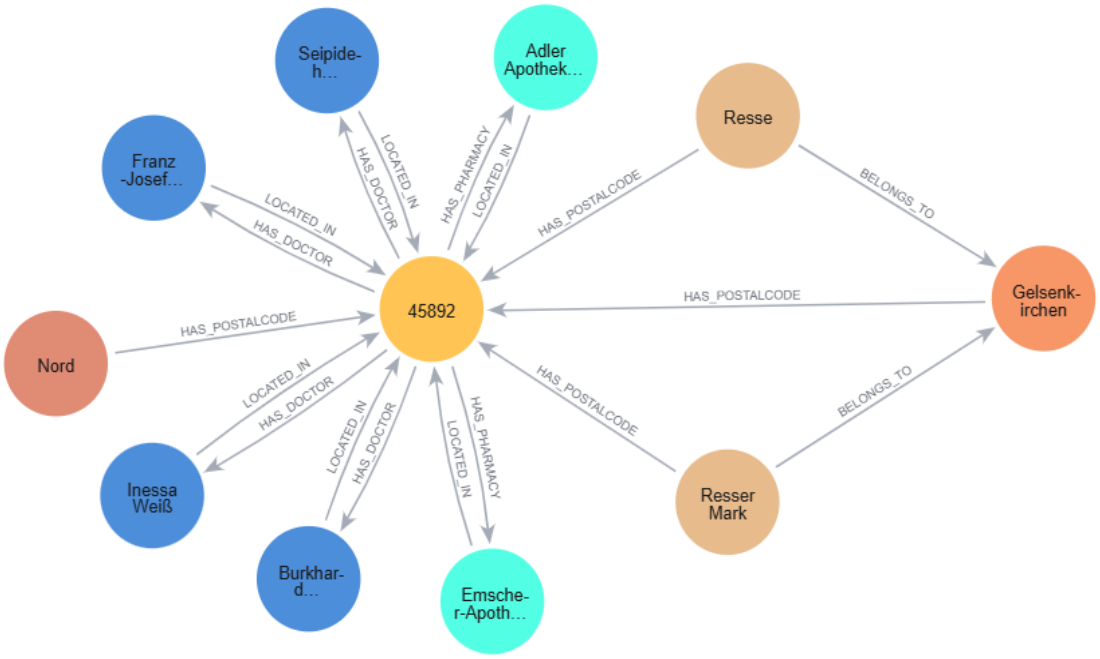
Neben stark zusammenhängenden Strukturen kann man gerichtete Graphen auch ohne Beachtung der Kantenrichtung betrachten. Dadurch entsteht das Konzept der schwach zusammenhängenden Graphen, das im Folgenden erklärt wird.

2.3.2. Schwach zusammenhängende Graphen

Ein gerichteter Graph G= (V, E) heißt schwach zusammenhängend, wenn sein zugrunde liegender ungerichteter Graph zusammenhängend ist. Das bedeutet, dass der Graph auch nach dem Ignorieren der Kantenausrichtungen weiterhin als zusammenhängend betrachtet werden kann. Jeder stark zusammenhängende Graph ist auch schwach zusammenhängend, die Umkehrung gilt jedoch nicht (Heun, 2003, S. 161).

Abbildung 9

Darstellung eines schwach zusammenhängenden Graphen



In Abbildung 9 ist ein schwach verbundener Graph dargestellt, der die Beziehungen zwischen dem Postleitzahlknoten *45892* und den zugehörigen Knoten wie Stadtteilen, Arztpraxen, Apotheken und der Stadt veranschaulicht.

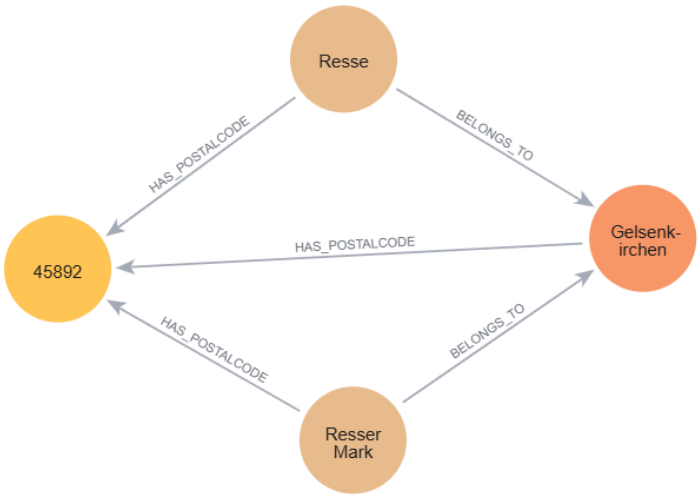
Eigenschaften schwach verbundener Graphen:

- **Ungerichtete Konnektivität:** Wird die Richtung der Kanten ignoriert, besteht eine Verbindung zwischen allen Knoten (Heun, 2003, S. 161).
  In Abbildung 9 ist zu erkennen, dass eine Verbindung zwischen allen Knoten besteht, wenn die Richtung der Kanten ignoriert wird.

- **Mehrere Komponenten möglich:** Ein gerichteter Graph kann mehrere schwach zusammenhängende Komponenten aufweisen, wenn in der ungerichteten Betrachtung nicht alle Knoten miteinander verbunden sind. Die Kantenrichtung spielt dabei keine Rolle, da sie bei der Bestimmung schwach zusammenhängender Komponenten ignoriert wird. Das Konzept der schwach zusammenhängenden Komponenten wird im folgenden Unterkapitel näher erläutert.
- **Keine vollständige Erreichbarkeit:** Aufgrund der Kantenrichtung sind bestimmte Knoten nicht in beide Richtungen erreichbar.
  Im gezeigten Abbildung 9 ist zu erkennen, dass Knoten wie Stadtteile und Stadtbezirke nicht vollständig erreichbar sind, da ihre Beziehungen nur in eine Richtung verlaufen und keine Rückverbindungen bestehen.
- **Zyklen nur im ungerichteten Sinn:** Im ungerichteten Graphen können Zyklen entstehen, während dies im gerichteten Fall nicht notwendigerweise der Fall ist. In Abbildung 10 ist ein schwach verbundener Graph dargestellt, in dem Zyklen nur im ungerichteten Sinn auftreten.

Abbildung 10

Schwach verbundener Graph mit Zyklen



Beispiele für Zyklen im ungerichteten Sinn:

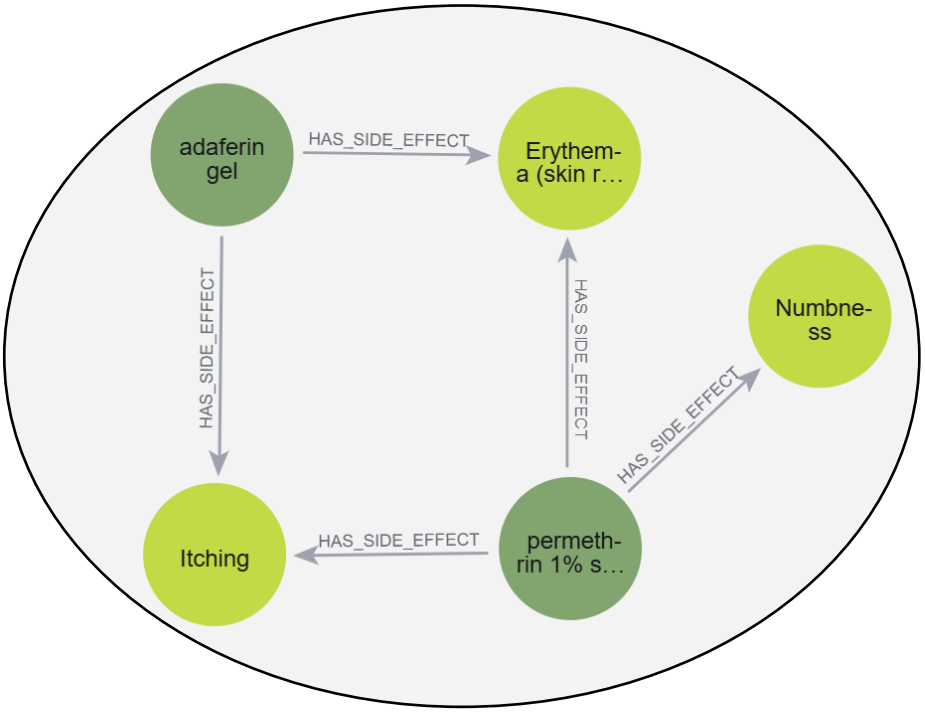
- *45892 – Resser Mark – Gelsenkirchen*
- *45892 – Resse – Gelsenkirchen*

Schwach zusammenhängende Komponenten

Schwach zusammenhängende Komponenten sind Teilgraphen eines gerichteten Graphen, bei denen alle Knoten miteinander verbunden sind, wenn man die Richtung der Kanten ignoriert (Heun, 2003, S. 161). Schwach zusammenhängende Komponenten sind in vielen Bereichen von Bedeutung. In Kommunikationsnetzen gewährleisten sie, dass alle Knoten erreichbar bleiben, auch wenn die Richtung der Kanten ignoriert wird. In sozialen Netzwerken können sie Gruppen von Nutzern darstellen, die indirekt miteinander verbunden sind. Darüber hinaus werden Connected Components häufig in der Betrugserkennung (Fraud Detection) eingesetzt. Durch die Identifikation von Gruppen, in denen plötzlich neue Knoten oder Verbindungen auftreten, lassen sich ungewöhnliche Muster frühzeitig erkennen (Needham & Hodler, 2019, S. 124).

Abbildung 11

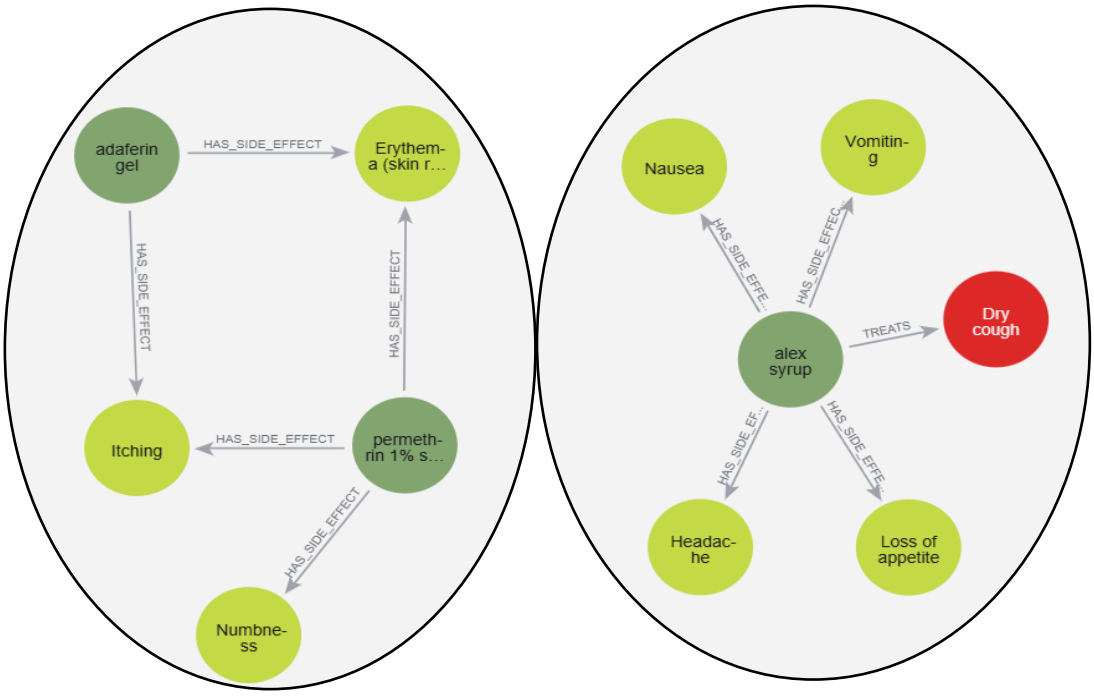
Schwach zusammenhängende Komponente eines Graphen, bestehend aus Arzneimittel und deren Nebenwirkungen.



In Abbildung 11 ist eine schwach zusammenhängende Komponente eines Graphen dargestellt. Sie besteht aus den Nebenwirkungs-Knoten *Erythema (skin redness), Numbness, Itching* sowie den Arzneimittel-Knoten *Permethrin 1% soap* und *Adaferin gel.*

Abbildung 12

Zwei schwach zusammenhängende Komponenten bestehend aus Arzneimittel-, Nebenwirkungen- und Krankheits-Knoten.



In Abbildung 12 zeigt die linke Komponente die Beziehungen zwischen Medikamenten und ihren Nebenwirkungen. Die rechte Komponente enthält zusätzlich eine Krankheit, die durch ein bestimmtes Medikament behandelt wird.

Zwei vorhandene schwach zusammenhängende Komponenten zeigen, dass der Graph aus voneinander getrennten Teilstrukturen besteht, zwischen denen keine Kante existiert. Wenn ein Graph mehr als eine schwach zusammenhängende Komponente hat, ist der Graph nicht zusammenhängend. Eine Traversierung zwischen diesen Komponenten ist daher nicht möglich (Bondy & Murty, 2008, S. 5).

2.4.Neo4j und die Cypher-Abfragesprache

Die technische Umsetzung dieser Bachelorarbeit erfolgt mithilfe der Graphdatenbank Neo4j. Neo4j ist eine native Graphdatenbank, bei der das zugrunde liegende Graphmodell bis auf die Speicherebene abgebildet wird. Dadurch können Knoten und Beziehungen direkt und ohne zusätzliche Überschicht gespeichert und verarbeitet werden. Das System unterstützt ACID-konforme Transaktionen und bietet Optionen für den Betrieb in verteilten Umgebungen, beispielsweise durch Clustering. ACID bedeutet, dass die Datenbank Daten sauber, korrekt und stabil verarbeitet, auch wenn mehrere Vorgänge gleichzeitig laufen oder ein Fehler auftritt. Für

die Abfrage der Daten wird die deklarative Sprache **Cypher** verwendet. Sie orientiert sich in ihrer Syntax an SQL, ist jedoch speziell auf die Modellierung und Analyse von Graphstrukturen ausgelegt. Neo4j ermöglicht zudem die Anbindung an verschiedene Programmiersprachen, darunter Java, Python und JavaScript, wodurch eine Integration in unterschiedliche Entwicklungsumgebungen ohne erheblichen Zusatzaufwand möglich ist (Neo4J, o. D.).

Cypher-Abfragesprache

Datenbankabfragen bilden die Grundlage der Interaktion zwischen Benutzer und Datenbank. Sie dienen dazu, benötigte Informationen zu suchen, zu filtern oder zu verändern. Eine Datenbankabfrage kann als eine Anfrage nach Daten aus einer Datenbank definiert werden. Zur Formulierung von Datenbankabfragen nutzt Neo4j die deklarative Abfragesprache Cypher.

- **Cypher** ist eine einfache und zugleich ausdrucksstarke Abfragesprache für Neo4j Graphdatenbank. Mit Cypher lassen sich Muster im Graphen beschreiben, um gezielt nach bestimmten Daten zu identifizieren und zu modifizieren. Die Schreibweise orientiert sich dabei an der Art, wie man Graphen typischerweise auf einem Whiteboard darstellt (Robinson et al., 2015, S. 27–29).

Abbildung 13

Cypher-Abfrage mit MATCH-, WHERE- und RETURN-Klausel

```
MATCH (d:Disease)
WHERE d.name = "Hypertension (high blood pressure)"
RETURN d;
```

In Abbildung 13 ist ein Beispiel für eine einfache Cypher Abfrage dargestellt. Die Abfrage sucht nach der Krankheitsknoten, deren Name *„Hypertension (high blood pressure)"*.

- Die **MATCH-Klausel** in Cypher dient dazu, bestimmte Muster innerhalb der Graphstruktur zu definieren, nach denen die Datenbank suchen soll. Dabei können sowohl Knoten, Beziehungen als auch deren Eigenschaften angegeben werden. Auf diese Weise lassen sich gezielt Abfragen formulieren, die den Graphen durchlaufen und die für die Analyse relevanten Daten zurückliefern. (NEO4J docs, o. D.c).

  neben der MATCH-Klausel stellt Cypher eine Reihe weiterer grundlegender Klauseln zur Verfügung, die für den Aufbau und die Verarbeitung von Abfragen entscheidend sind:

- **RETURN**: Legt fest, welche Knoten, Beziehungen oder Eigenschaften aus den gefundenen Mustern an den Client zurückgegeben werden sollen. (Robinson et al., 2015, S. 30–31).
- **WHERE**: Dient der Formulierung von Bedingungen, um die Ergebnisse einer Mustersuche gezielt zu filtern. (Robinson et al., 2015, S. 31).
- **CREATE** Erzeugen neue Knoten und Beziehungen im Graphen. (Robinson et al., 2015, S. 31).

Abbildung 14

Beispiel einer CREATE Abfrage

```
CREATE (p:PostalCode {code: "45899"})
RETURN p;
```

- **MERGE**: Gewährleistet, dass ein bestimmtes Muster im Graphen existiert, indem entweder bestehende Elemente wiederverwendet oder – falls nicht vorhanden – neue erzeugt werden (Robinson et al., 2015, S. 31).

Abbildung 15

Beispiel einer MERGE-Abfrage.

```
MATCH (d:District {name: "West"})
MERGE (p:PostalCode {code: "45889"})
MERGE (p)-[:LOCATED_IN]->(d)
RETURN p, d;
```

In Abbildung 15 ist ein Beispiel für die Verwendung von MERGE dargestellt. Die Abfrage verbindet den Postleitzahl-Knoten *45889* mit dem Stadtbezirk West. Existiert der entsprechende Postleitzahl-Knoten oder LOCATED_IN Beziehung noch nicht, wird er automatisch angelegt.

- **DELETE**: Entfernt Knoten, Beziehungen oder Eigenschaften aus dem Graphen (Robinson et al., 2015, S. 31).

Abbildung 16

Beispiel einer DELETE Abfrage

```
MATCH (p:Pflegedienst {name: "Connys Hilfe"})
DELETE p;
```

In Abbildung 16 ist eine einfache Löschabfrage dargestellt. Der gezeigte Cypher-Befehl wird der Knoten des Pflegedienstes *Connys Hilfe* aus dem Graphen entfernt.

- **LOAD CSV** ist ein Cypher-Befehl in Neo4j, mit dem Daten aus einer CSV-Datei in eine bereits laufende Datenbank geladen werden können. Er wird verwendet, um Daten schrittweise zu importieren, zu aktualisieren oder zu erweitern, ohne die Datenbank neu aufzubauen. (Robinson et al., 2015, S. 100–101).

Abbildung 17

Beispiel einer LOAD CSV Abfrage

```
LOAD CSV WITH HEADERS FROM 'file:///pharmacies.csv' AS row
MERGE (p:Pharmacy {
    name: row.name,
    street: row.`addr:street`,
    housenumber: row.`addr:housenumber`,
    postcode: row.`addr:postcode`
})
SET p.city = row.`addr:city`
```

In Abbildung 17 ist ein Beispiel für die Verwendung des Befehls LOAD CSV dargestellt. Mit diesem Cypher-Befehl werden Apothekenknoten aus einer CSV-Datei in die Datenbank importiert und deren Eigenschaften, wie Name, Straße, Hausnummer, Postleitzahl und Stadt, gesetzt.

- **SET**: Ermöglicht das Setzen oder Aktualisieren von Eigenschaften eines Knotens oder einer Beziehung (Robinson et al., 2015, S. 31).

Im gezeigten Beispiel aus Abbildung 17 wird mit dem Befehl SET p.city = row.addr:city die Eigenschaft Stadt des Knotens Apotheke auf den entsprechenden Wert aus der CSV-Datei gesetzt.

- **WITH**: Der WITH-Befehl dient dazu, verschiedene Teile einer Cypher-Abfrage miteinander zu verbinden. Er ermöglicht es, Zwischenergebnisse explizit an den folgenden Abfrageteil weiterzugeben und dadurch Abfragen strukturiert, schrittweise und logisch aufzubauen (Robinson et al., 2015, S. 31).

Abbildung 18

Beispiel einer WITH-Abfrage

```
MATCH (p:Pharmacy)
WITH p.postcode AS postleitzahl, count(p) AS anzahl
WHERE anzahl > 5
RETURN postleitzahl, anzahl
ORDER BY anzahl DESC;
```

In Abbildung 18 wurde eine Gruppierung der Apotheken nach Postleitzahl dargestellt. Mithilfe der WITH-Klausel werden die Apothekenknoten nach ihrer Eigenschaft Postleitzahl gruppiert und gezählt. Durch die anschließende WHERE-Bedingung werden nur jene Postleitzahlen berücksichtigt, in denen mehr als fünf Apotheken vorhanden sind.

2.5. Analyseverfahren in Graphdatenbanken

Für die Analyse eines Graphen gibt es verschiedene grundlegende Methoden. Mit ihnen kann man zum Beispiel herausfinden, wie weit Knoten voneinander entfernt sind, wie gut das Netzwerk insgesamt verbunden ist oder welche Knoten besonders wichtig sind. Diese Analyseverfahren helfen dabei, den Graphen besser zu verstehen und werden auch in dieser Arbeit genutzt, um die medizinischen und geografischen Daten auszuwerten.

Kürzester Pfad (Shortest-Path)

Die Bestimmung kürzester Pfade zählt zu den am häufigsten durchgeführten Aufgaben im Bereich der Graphalgorithmen und bildet die Grundlage für zahlreiche weiterführende Analysen. Unter einem kürzesten Pfad versteht man dabei die Traversierung zwischen zwei Knoten, die entweder die geringste Anzahl an Kanten oder das niedrigste Gewicht aufweist. In gerichteten Graphen bezieht sich dies auf den kürzesten Pfad unter Berücksichtigung der vorgegebenen Richtungen der Kanten (Needham & Hodler, 2019, S. 27). Beispiele für Anwendungsfälle des Shortest-Path-Algorithmus sind etwa die Bestimmung von Routen zwischen Orten, wie sie in Web-Mapping-Diensten (z. B. Google Maps) eingesetzt wird, oder die Berechnung von Verbindungen zwischen Personen in sozialen Netzwerken. So zeigt beispielsweise LinkedIn an, über wie viele Zwischenpersonen man mit einem anderen Nutzer verbunden ist und listet dabei auch gemeinsame Kontakte auf (Needham & Hodler, 2019, S. 50).

Zur Bestimmung des nächstgelegenen Krankenhauses wurde die kürzeste Verbindung zwischen dem ausgewählten Stadtteil Heßler und allen vorhandenen Krankenhäusern berechnet. Die entsprechende Cypher-Abfrage ist in Abbildung 19 zu sehen.

Abbildung 19

Cypher-Abfrage zur Bestimmung der Kürzester Pfad

```
MATCH (st:SubDistrict {name: "Heßler"})
MATCH (h:Hospital)
MATCH path = shortestPath((st)-[*1..10]->(h))
RETURN
    h.name AS hospital_name,
    length(path) AS distance
ORDER BY distance ASC
LIMIT 1;
```

Die Abfrage liefert das Ergebnis *St. Josef-Hospital* mit einer Distanz von 2. Da der Graph keine Kantengewichte enthält, wurde der kürzeste Pfad anhand der Anzahl der durchlaufenen Kanten bestimmt. Der entsprechende Pfad ist in Abbildung 21 dargestellt.

Abbildung 20

Darstellung des kürzesten Pfades vom Heßler zum St. Josef-Hospital

Längster kürzester Pfad (Graphdurchmesser)

Der längste kürzeste Pfad auch als Durchmesser eines Graphen bezeichnet, gibt den maximalen Wert aller kürzesten Wege zwischen Knotenpaaren an. Er zeigt somit an, welche Knoten im Netzwerk am weitesten voneinander entfernt sind, selbst wenn der jeweils optimale Weg gewählt wird. Dies lässt sich anhand eines Verkehrsnetzes veranschaulichen: Der Durchmesser zeigt beispielsweise jene Straßenbahnhaltestellen an, die trotz Nutzung des optimalen Verbindungswegs die größte Entfernung zueinander aufweisen oder bei denen auf dem kürzesten Weg besonders viele Zwischenhaltestellen durchlaufen werden müssen (Needham & Hodler, 2019, S. 27).

Zentralitätsmaße

Unter Zentralitätsmaße versteht man in der Netzwerkanalyse ein Maß zur Bestimmung der Relevanz einzelner Knoten innerhalb eines Graphen. Dabei wird untersucht, welche Knoten eine besonders einflussreiche Position im Netzwerk einnehmen, sei es durch ihre Erreichbarkeit, ihre Rolle bei der Weitergabe von Informationen oder ihre Funktion als verbindendes Element zwischen Gruppen. Es existieren verschiedene Zentralitätsmaße, wie etwa Zwischenzentralität, Nahzentralität und Gradzentralität (Needham & Hodler, 2019, S. 77). In dieser Arbeit wird die Gradzentralität verwendet und im folgenden Abschnitt als Grundlage näher erläutert.

Gradezentralität

Die Gradzentralität beschreibt, wie viele direkte Verbindungen ein Knoten zu anderen Knoten im Netzwerk hat. Je mehr Verbindungen ein Knoten besitzen, desto stärker ist er vernetzt und desto wichtiger kann seine Rolle im Informations- oder Beziehungsfluss sein. Dieses Maß wird in vielen Bereichen genutzt. In sozialen Netzwerken lässt sich damit zum Beispiel erkennen, welche Personen besonders viele Kontakte haben und deshalb als einflussreich gelten. Prominente Nutzerinnen und Nutzer auf Plattformen wie Twitter haben häufig eine hohe Gradzentralität. Auch in der Betrugserkennung ist dieses Maß hilfreich: Auf Online-Auktionsseiten fällt oft auf, dass betrügerische Accounts ungewöhnlich viele Verbindungen zu anderen Nutzern haben, weil sie zusammenarbeiten, um Preise zu manipulieren. Eine hohe Gradzentralität kann daher ein Hinweis auf auffälliges oder betrügerisches Verhalten sein (Needham & Hodler, 2019, S. 81–84).

3. Datenbankaufbau und Herausforderungen

In diesem Kapitel wird beschrieben, wie die Datenbank für diese Arbeit Schritt für Schritt aufgebaut wurde. Dazu gehören die Auswahl und Beschaffung der Daten, ihre Bereinigung und Aufbereitung sowie die Entwicklung des Datenbankschemas. Außerdem werden die Herausforderungen erläutert, die während dieses Prozesses aufgetreten sind und wie diese gelöst wurden. Dadurch wird deutlich, wie die finale Struktur des Graphen entstanden ist.

3.1. Ziel des Datenbankaufbaus

Das Ziel des Datenbankaufbaus war es, einen realen Datensatz mit medizinischen und geografischen Informationen so zu strukturieren, dass sowohl die Eigenschaften des Graphen als auch praktische Anwendungsfälle untersucht werden können. Dabei sollte die Datenbank verschiedene Bereiche miteinander verbinden, zum Beispiel:

- geografische Daten (Stadt, Stadtbezirke, Stadtteile, Postleitzahlen)
- medizinische Informationen (Krankheiten, Symptome, Medikamente, Nebenwirkungen)
- Versorgungseinrichtungen (Krankenhäuser, Apotheken, Arztpraxen, Rettungsdienste, Pflegeeinrichtungen)
- statistische Werte (Einwohnerzahlen, Geburten, Sterbefälle, COVID-19-Daten)

Der Aufbau des Modells musste flexibel sein, damit unterschiedliche Datentypen zuverlässig miteinander verknüpft werden können.

3.2. Aufbau der Entwicklungsumgebung

Für die Umsetzung der Arbeit wurde die Graphdatenbank **Neo4j** zusammen mit der Entwicklungsumgebung **PyCharm** verwendet. Neo4j Desktop (Version 2.0.4) diente dabei als lokale Arbeitsumgebung und stellte die Abfragesprache **Cypher** in Version 5 zur Verfügung. Zusätzlich wurde das **Graph Data Science Framework (GDS)** in Version 2.22.0 installiert, um fortgeschrittene Analyseverfahren wie SCC, WCC oder allShortestPaths ausführen zu können.

Für die Vorbereitung der Daten kamen verschiedene Python-Bibliotheken zum Einsatz, darunter **pandas**, **geopandas** und **osmnx**. Mit diesen Werkzeugen konnten Geodaten aus OpenStreetMap abgefragt, bearbeitet und anschließend als CSV-Dateien für den Import in Neo4j gespeichert werden.

3.3. Datenbeschaffung und Probleme bei den Quellen

Für den Aufbau der Datenbank wurden verschiedene öffentliche und wissenschaftliche Datenquellen verwendet. Da diese Daten aus unterschiedlichen Systemen stammten und unterschiedlich strukturiert waren, war die Zusammenführung teilweise anspruchsvoll.

Ein großer Teil der Informationen stammt aus den Open-Data-Portalen der Städte Gelsenkirchen und Essen. Dazu gehören Daten zu Krankenhäusern, Pflegeeinrichtungen, Geburten, Sterbefällen, Einwohnerzahlen, Defibrillatoren und Rettungsdiensten. Diese Daten lagen überwiegend als CSV-Dateien vor. Eine besondere Herausforderung war, dass Gelsenkirchen deutlich mehr und vollständigere Daten zur Verfügung stellte als Essen (Stadt Essen, 2025; Stadt Gelsenkirchen, 2025).

Zusätzlich zu den über Open-Data-Portale bezogenen Datensätzen wurden Informationen zu Apotheken und Arztpraxen direkt aus OpenStreetMap (OSM) extrahiert (OpenStreetMap Foundation, 2025). Abbildung 21 zeigt beispielhaft den verwendeten Python-Code zur Extraktion der Apotheken in Gelsenkirchen.

Abbildung 21

Python-Code zur Extraktion der Apotheken in Gelsenkirchen.

```python
import osmnx as ox
import pandas as pd

def extract_pharmacies(city="Gelsenkirchen, Germany", output_file="pharmacies.csv"):
    tags = {"amenity": "pharmacy"}
    gdf = ox.features_from_place(city, tags)
    cols = ["osmid", "name", "addr:street", "addr:housenumber",
            "addr:postcode", "addr:city"]
    gdf["lon"] = gdf.centroid.x
    gdf["lat"] = gdf.centroid.y
    gdf[cols + ["lat", "lon"]].to_csv(output_file, index=False)
```

Zusätzlich wurden medizinische Daten aus zwei wissenschaftlichen Quellen Kaggle und Zenodo genutzt. Diese lieferten Informationen zu Krankheiten, Symptomen, Medikamenten und Nebenwirkungen. Da die Datensätze jedoch sehr umfangreich waren und viele für diese Arbeit nicht benötigte Informationen enthielten, wurden sie auf die relevanten Einträge reduziert und vereinheitlicht (Ashariana, 2021; Shudhanshu et al., 2022). Die COVID-19-Statistiken für Essen und Gelsenkirchen stammen aus dem öffentlich zugänglichen Portal *corona-in-zahlen.de*, (Siekmann, 2025a, 2025b).

Nachdem alle Datenquellen aufbereitet und geprüft wurde, konnte mit dem Bereinigungs- und Modellierungsprozess begonnen werden. Dieser wird im folgenden Kapitel beschrieben.

3.4. Datenaufbereitung und Transformation

Bevor die Daten in Neo4j importiert werden konnten, mussten sie vereinheitlicht und bereinigt werden. Zu den wichtigsten Schritten gehörten:

- Formatierung und Strukturierung der CSV-Dateien
- Entfernen fehlerhafter oder unvollständiger Zeilen
- Vereinheitlichung von Labels und Schreibweisen
- Zuordnung medizinischer Entitäten zu passenden Kategorien

Während dieser Aufbereitung traten mehrere praktische Probleme auf. Ein zentrales Problem war der Umgang mit Sonderzeichen wie ä, ö, ü oder ß. Einige CSV-Dateien konnten diese Zeichen nicht korrekt einlesen, was zu fehlerhaften Namen und Einträgen führte. Diese Fehler wurden überwiegend mit Microsoft Excel vor dem Import manuell korrigiert.

Ein weiteres Beispiel betrifft die Arztpraxen. In den Datensätzen wurden unterschiedliche Titel wie Dr oder Dr. med verwendet. Diese uneinheitliche Schreibweise führte beim Import dazu, dass manche Praxen nicht eindeutig erkannt wurden. Da eine manuelle Bereinigung zu aufwendig gewesen wäre, wurde die Cypher-Abfrage so angepasst, dass diese Titel beim Import automatisch entfernt werden. Auf diese Weise konnte zunächst eine funktionsfähige Datenbank aufgebaut werden.

Nach der Bereinigung konnten die Daten in Neo4j importiert werden. Ein Teil der Daten unter anderem Apotheken und Arztpraxen wurde mit dem Befehl LOAD CSV eingelesen. Die grundlegende Funktionsweise dieses Imports wurde bereits im Grundlagenkapitel (Abbildung 18) erläutert. Deshalb wird der Import an dieser Stelle nicht erneut beschrieben.

In diesem Abschnitt werden die zusätzlichen Schritte dargestellt, die nach dem Import notwendig waren. Dazu gehörte zum Beispiel das manuelle Erstellen der Beziehungen zwischen Apotheken und den passenden Postleitzahlen. Die verwendete Cypher-Abfrage sowie die Visualisierung der erzeugten Knoten und Beziehungen sind in den Abbildungen 22 und 23 dargestellt.

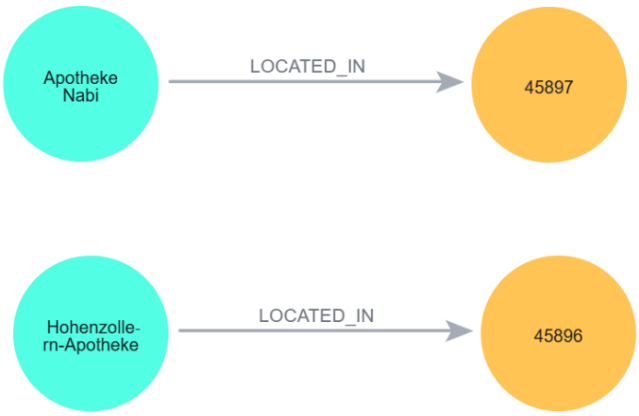
Abbildung 22

Cypher-Abfrage zur Erstellung der Beziehung von Apotheken zu Postleitzahlen

```
MATCH (p:Pharmacy), (po:PostalCode)
WHERE p.postcode = po.code
MERGE (p)-[:LOCATED_IN]->(po);
```

Abbildung 23

Beispiele von Apotheken- und Postleitzahl-Knoten mit Beziehungen



Nachdem alle Daten erfolgreich bereinigt und getestet wurden, begann die Modellierung des Datenbankschemas. Der Modellierungsprozess wird in Kapitel 3.5 beschrieben.

3.5. Modellierungsprozess: Von erster Idee zum finalen Graphen

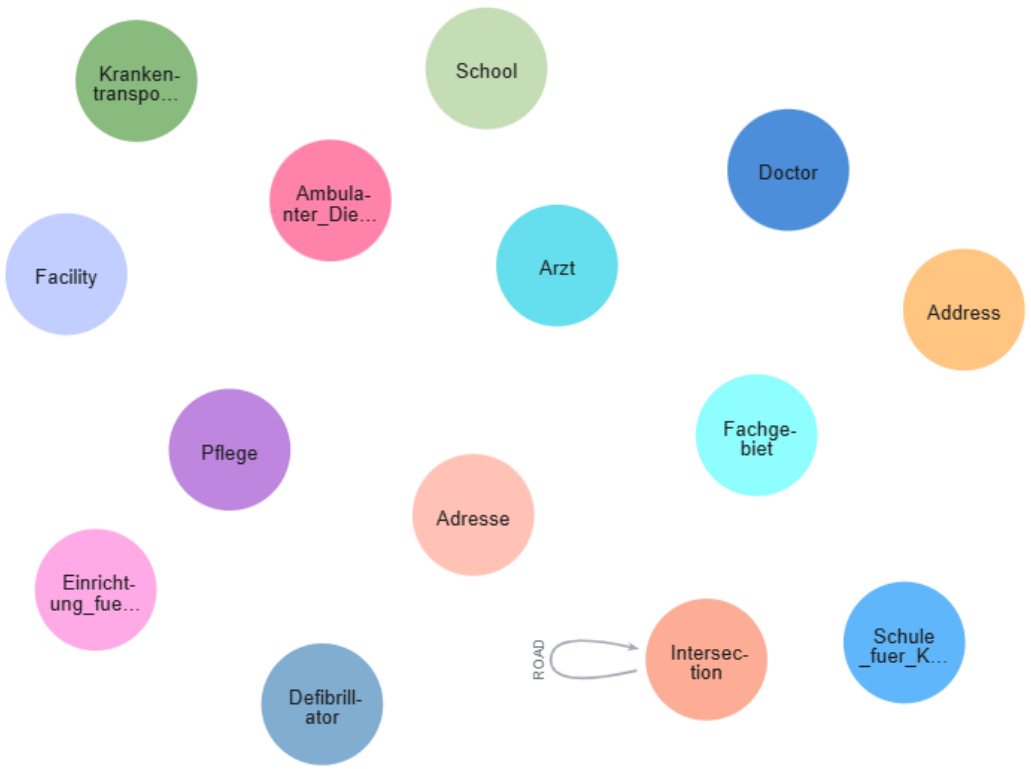
Der Aufbau des Datenbankschemas erfolgte schrittweise. Im Verlauf des Prozesses wurden verschiedene Modellierungsansätze entwickelt, getestet und teilweise wieder verworfen. Dieses Kapitel zeigt den Weg vom ersten Konzept bis zum finalen, einsatzbereiten Graphen auf und erläutert, welche Überlegungen und Herausforderungen die Modellierung beeinflusst haben.

3.5.1. Erster Entwurf des Modells

Ursprünglich war geplant, zusätzlich das Straßennetz in das Datenbankschema zu integrieren. Der erste Entwurf enthielt daher die Knoten *Intersection* und verschiedene Einrichtungen wie Krankenhäuser, Pflegeeinrichtungen und Arztpraxen. Ziel war es, Wege zwischen medizinischen Einrichtungen oder zwischen Einrichtungen und Adressen berechnen zu können. Dafür wurden Straßen über die Beziehung *ROAD* modelliert. Die frühe Version dieses Modells ist in Abbildung 24 dargestellt.

Abbildung 24

Erster Modellierungsentwurf mit Straßen- und Knotenpunkten



Während der Entwicklung stellte sich heraus, dass diese Modellierung schnell zu groß wurde. Diese Version umfasste bereits **5024 Knoten und 10 744 Kanten**. Ein Großteil davon entstand durch die vielen Straßen- und Kreuzungsknoten und Kanten. Da diese Daten keinen Nutzen für die späteren Analysen brachten und die Datenbank nur unnötig vergrößert hätten, wurde dieser Ansatz verworfen.

3.5.2. Zweiter Entwurf des Modells

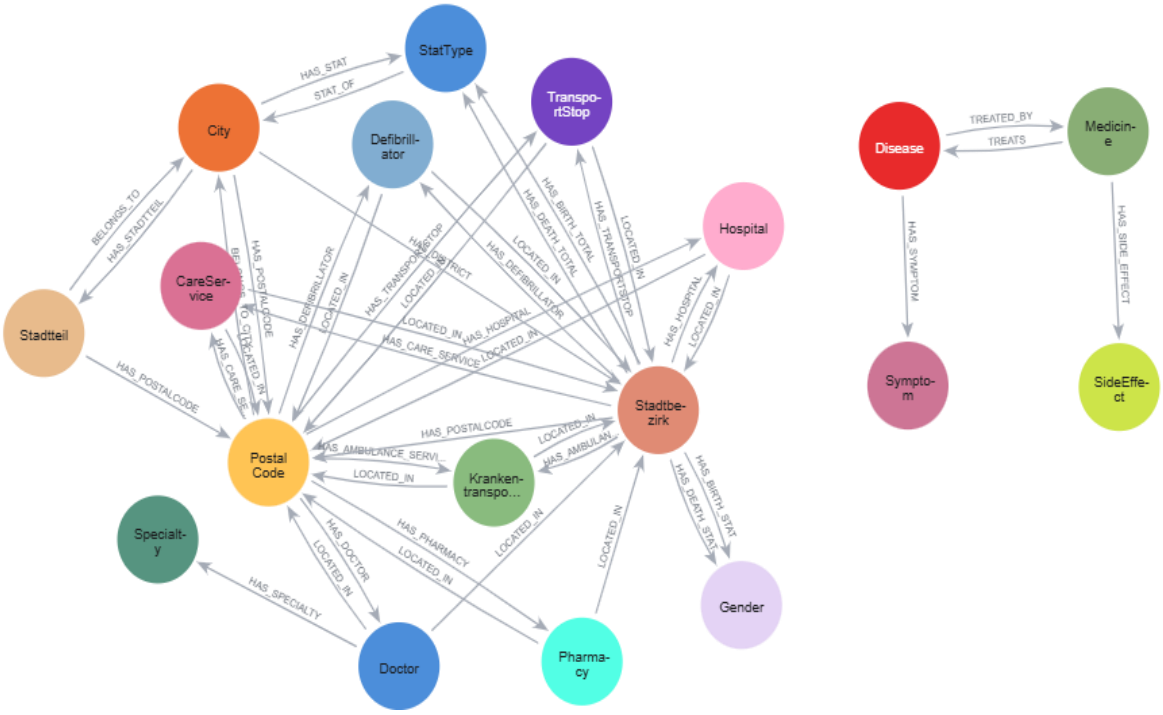
Nach der Entscheidung, das Straßennetz nicht weiter zu berücksichtigen, wurde ein neuer Entwurf entwickelt. Dieser orientierte sich zum ersten Mal an einer klaren Trennung der drei zentralen Datenbereiche:

1. **Geografische Struktur**
2. **Medizinische Struktur**
3. **Versorgungs- und Statistikdaten**

Dieser zweite Entwurf ist in Abbildung 25 dargestellt.

Abbildung 25

Zweiter Modellierungsentwurf der Graphdatenbank



Allerdings fehlten in diesem Entwurf noch mehrere wichtige Daten, zum Beispiel die COVID-19-Statistiken oder die Beziehungen zwischen Krankheiten und Fachrichtungen. Zudem wurde die Stadt Essen in diesem Entwurf noch nicht berücksichtigt, da die Datensuche und Aufbereitung für diesen Bereich zu diesem Zeitpunkt noch nicht abgeschlossen war.

Im finalen Schema wurden die zuvor fehlenden Daten ergänzt, darunter die Stadt Essen, das Bundesland Nordrhein-Westfalen, die Darreichungsformen der Medikamente sowie die vollständigen COVID-19-Statistiken.

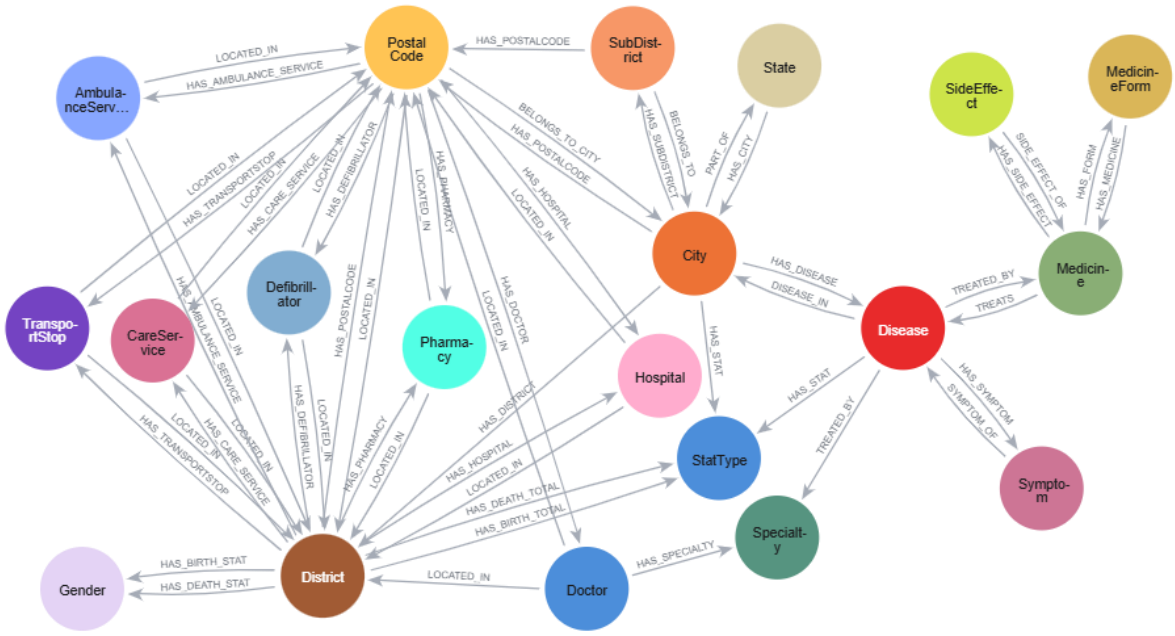
3.5.3. Finaler Entwurf des Schemas

Auf Grundlage der offenen Punkte aus dem zweiten Entwurf wurde das Datenbankschema weiterentwickelt und schrittweise erweitert. Dabei wurden fehlende Informationen ergänzt, Strukturen vereinheitlicht und die Datenbereiche miteinander verbunden. In dieser Phase wurden unter anderem die zuvor noch fehlenden Daten integriert, wie die Stadt Essen, das Bundesland Nordrhein-Westfalen, die Darreichungsformen der Medikamente sowie die vollständigen COVID-19-Statistiken.

Das Ergebnis dieser Überarbeitung ist das finale Datenbankschema, das in Abbildung 26 dargestellt ist.

Abbildung 26

Visualisierung des finalen Datenbankschemas.



Nach Abschluss der Modellierungsarbeiten wurde die endgültige Version des Schemas implementiert. Der vollständig aufgebaute Graph umfasst **1123 Knoten** und **4487 Kanten**. Diese Größe entsteht durch die Zusammenführung aller medizinischen, geografischen und statistischen Datenquellen und bildet die Grundlage für die anschließenden Analysen.

Das finale Modell besteht aus:

- **20 Knotentypen**
- **31 Beziehungstypen**

4. Analyse der Graphstruktur und praktischer Anwendungsfälle

Dieses Kapitel untersucht die strukturellen Eigenschaften und die praktischer Anwendungsfälle der aufgebauten Graphdatenbank. Für jede Analyse werden zunächst das Ziel, dann die verwendete Methodik, die gewonnenen Ergebnisse sowie deren Interpretation und schließlich die Diskussion dargestellt.

Für die Analyse der zusammenhängenden Komponenten sowie des Durchmessers wurde die GDS-Graphprojektion *myGraph* erstellt. Diese Projektion enthält alle Knoten und gerichteten Kanten des aufgebauten Graphen und bildet damit die vollständige Struktur des Modells ab (NEO4J docs, o. D.b). Auf dieser Grundlage wurden anschließend die einzelnen Analysen durchgeführt.

4.1. Analyse der stark und schwach zusammenhängenden Komponenten

Ziel dieser Analyse ist es, die Struktur des Datenbanksystems zu untersuchen und zusammenhängende Bereiche zu identifizieren. Durch die Bestimmung stark und schwach zusammenhängender Komponenten lässt sich beurteilen, wie gut der Graph vernetzt ist, welche Teile isoliert bleiben und ob die Modellierung insgesamt konsistent umgesetzt wurde. Zudem können auf diese Weise mögliche Import- oder Modellierungsfehler sichtbar werden.

4.1.1. Methodik

Zur Ermittlung der Komponenten wurde die myGraph-Projection des Gesamtdatenbestands genutzt. Anschließend wurden die GDS-Algorithmen *gds.scc.stream()* und *gds.wcc.stream()* auf die Graphprojektion angewendet.

Die verwendeten Cypher-Abfragen sind in Abbildung 27 für stark zusammenhängenden Komponente und 28 für schwach zusammenhängenden Komponente dargestellt.

Abbildung 27

Cypher Abfrage zur Ermittlung der stark zusammenhängenden Komponente

```
CALL gds.scc.stream('myGraph')
YIELD nodeId, componentId
WITH componentId, gds.util.asNode(nodeId) AS n
WITH componentId,
     collect(DISTINCT labels(n)) AS LabelGruppen,
     count(n) AS AnzahlKnoten
RETURN componentId,
       LabelGruppen AS Labels,
       AnzahlKnoten
ORDER BY AnzahlKnoten DESC;
```

Abbildung 28

Cypher Abfrage zur Ermittlung der schwach zusammenhängenden Komponente

```
CALL gds.wcc.stream('myGraph')
YIELD nodeId, componentId
WITH componentId, gds.util.asNode(nodeId) AS n
WITH componentId,
     collect(DISTINCT labels(n)) AS LabelGruppen,
     count(n) AS AnzahlKnoten
RETURN componentId,
       LabelGruppen AS Labels,
       AnzahlKnoten
ORDER BY AnzahlKnoten DESC;
```

Die gezeigten Abfragen wurden verwendet, um die Struktur des Graphen systematisch zu untersuchen. Mithilfe von gds.scc.stream() und gds.wcc.stream() werden die stark bzw. schwach zusammenhängenden Komponenten berechnet. Beide Prozeduren geben für jeden Knoten eine Komponenten-ID zurück, sodass erkennbar ist, welche Knoten zur gleichen verbundenen Gruppe gehören (NEO4J docs, o. D.d, o. D.e).

Im nächsten Schritt werden die internen Knoten-IDs mit gds.util.asNode() in vollständige Neo4j-Knoten umgewandelt, um deren Labels auslesen zu können. Anschließend werden alle Labeltypen einer Komponente mit collect(DISTINCT labels(n)) zusammengefasst und die Anzahl der enthaltenen Knoten über count(n) bestimmt. Abschließend erfolgt eine Sortierung der Komponenten nach ihrer Größe, sodass die größten und strukturell relevantesten Bereiche des Graphen zuerst sichtbar werden.

Dieselbe Vorgehensweise wird sowohl in der stark zusammenhängender Komponente-Analyse, bei der gerichtete Kanten berücksichtigt werden, als auch in der schwach zusammenhängender Komponente-Analyse, bei der die Richtung ignoriert wird, angewendet. Dadurch können sowohl eng verbundene Strukturen als auch lose oder isolierte Bereiche des Graphen identifiziert werden.

4.1.2. Ergebnisse

Die Analyse der Stark zusammenhängender Komponente ergab insgesamt 51 stark zusammenhängende Komponenten. Die größte Komponente enthält 1063 Knoten von insgesamt 1123 Knoten. Sie umfasst die wichtigsten Teile des Datensatzes, zum Beispiel Bundesland, Stadt, Stadtbezirk, Stadtteil, Postleitzahl, Haltestelle, Rettungsdienst, Krankenhaus, Pflegedienst, Defibrillator, Apotheke, Arztpraxis, Krankheit, Symptom, Medikament, Darreichungsform und Nebenwirkung.

Die zweitgrößte stark zusammenhängender Komponente besteht aus 11 Knoten. Eine zusätzliche Prüfung der enthaltenen Knoten zeigt, dass es sich dabei um die Krankheit *Conjunctivitis (Pink Eye)* und ihre Symptome handelt. Weitere 34 stark zusammenhängenden Komponente bestehen jeweils aus einzelnen Knoten von den Fachrichtungen.

Zusätzlich wurden zwei stark zusammenhängender Komponente identifiziert, die aus den Geschlechtsknoten *männlich* und *weiblich* bestehen und in den Bevölkerungsstatistiken der Stadtbezirke verwendet werden. Darüber hinaus existieren 13 stark zusammenhängender Komponente, die jeweils aus einzelnen Statistik-Knoten bestehen.

Die Analyse der schwach zusammenhängenden Komponenten zeigte, dass der Graph aus nur zwei schwach zusammenhängender Komponente besteht. Die größte davon enthält 1112 Knoten von insgesamt 1123 Knoten und umfasst nahezu alle Entitäten des Datenmodells. Die zweite schwach zusammenhängender Komponente besteht aus 11 Knoten und entspricht vollständig der isolierten *Conjunctivitis (Pink Eye)* -Komponente, die bereits in der stark zusammenhängender Komponente -Analyse als eigenständige Struktur erkennbar war.

4.1.3. Interpretation der Ergebnisse

Die Ergebnisse zeigen, dass der überwiegende Teil des Graphen gut vernetzt ist. Die große stark zusammenhängende Komponente bestätigt, dass geografische Daten, medizinische Einrichtungen, Krankheiten, Symptome und Versorgungsstrukturen eng miteinander verbunden sind.

Die stark zusammenhängende Komponente mit 11 Knoten der Krankheit *Conjunctivitis(Pink Eye)* entsteht, weil in den Quelldaten keine Medikamente für das Krankheit hinterlegt sind und die Symptome von keiner anderen Krankheit geteilt werden. Dies ist kein Modellierungsfehler, sondern ein Zeichen für eine begrenzte Datenbasis.

Die isolierten Fachrichtung-Knoten zeigen, dass keine ausgehende Verbindung aus Fachrichtungen zu Krankheiten oder Arztpraxen existieren. Dies liegt daran, dass reale Daten zu diesen Verknüpfungen fehlen.

Die zwei Gender- stark zusammenhängender Komponente entstehen aufgrund der Struktur der Geburten- und Sterbestatistiken, die jeweils geschlechtsspezifisch modelliert wurden.

Darüber hinaus bestehen die meisten weiteren Statistiken- stark zusammenhängender Komponente überwiegend aus COVID-19-Statistikdaten der Stadt Gelsenkirchen. Für jede einzelne Kennzahl zum Beispiel Todesfälle oder Intensivpatienten wurde ein eigener Statistikknoten mit einem eigenen Wert angelegt. und die Daten keine rückwärts Kanten existieren kann.

Die Analyse der schwach zusammenhängenden Komponenten zeigt, dass der Graph insgesamt sehr gut strukturell verbunden ist. Mit nur zwei schwach zusammenhängender Komponente wird deutlich, dass fast alle Knoten trotz unterschiedlicher Datenbereiche, Teil eines einzigen großen, zusammenhängenden Netzwerks sind. Die große schwach zusammenhängender Komponente umfasst nahezu den gesamten Datenbestand und bestätigt, dass die Modellierung der geografischen und medizinischen Daten eine zusammenhängende Gesamtstruktur erzeugt.

Die schwach zusammenhängende Komponente mit 11 Knoten zeigt, dass sie identisch zur stark zusammenhängenden Komponente gleicher Größe ist. Gleichzeitig bleibt diese Komponente vollständig vom übrigen Teil des Graphen isoliert.

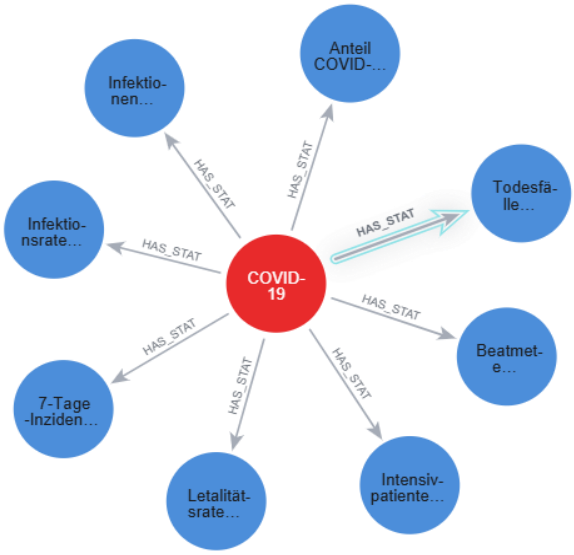
4.1.4. Diskussion

Die Struktur des Graphen ist grundsätzlich stabil und stark vernetzt. Die isolierte Conjunctivitis (Pink Eye) -Komponente deutet auf fehlende Daten zu Medikamenten hin. Die isolierte Conjunctivitis-(Pink Eye) -Komponente zeigt, dass in den Daten keine Informationen zu passenden Medikamenten vorhanden sind. Eine Verbindung zu Medikamenten hätte diese Krankheit in den größeren Graphen eingebunden. Obwohl keine Medikamente in der Datenbank vorliegen, wäre dies über die vorhandenen Darreichungsformen grundsätzlich möglich gewesen, dadurch die Analyse effizienter wird.

Eine weitere Einschränkung der Datenbank betrifft die uneinheitliche Modellierung der COVID-19-Statistiken. In Abbildung 29 ist die ursprüngliche Modellierung der COVID-19-Statistiken zu sehen, wie sie für die Stadt Gelsenkirchen umgesetzt wurde. Dabei wurden für jede einzelne Statistik beispielsweise *Todesfälle*, *7-Tage-Inzidenz* oder *Intensivpatienten* eigenständige Statistik-Knoten erstellt. Diese Statistik-Knoten wurden jedoch nicht mit dem Stadtknoten *Gelsenkirchen* verknüpft, sondern ausschließlich über **HAS_STAT**-Beziehungen direkt an den Krankheitsknoten *COVID-19* angebunden. Die eigentlichen statistischen Werte befinden sich dabei als Eigenschaften auf den jeweiligen Beziehungen.

Abbildung 29

Modellierung Covid 19 Statistik von Stadt Gelsenkirchen-1



Die Stadt Gelsenkirchen wurde anschließend korrekt über die Beziehung HAS_DISEASE mit dem COVID-19-Knoten verknüpft. Die zugehörigen COVID-19-Statistiken der Stadt wurden als Eigenschaften auf dieser Beziehung gespeichert.

Abbildung 30 zeigt diese Modellierung. Dort ist ebenfalls zu erkennen, dass zusätzlich eine Beziehung DISEASE_IN von COVID-19 zur Stadt existiert. Ursprünglich war jedoch vorgesehen, eine gerichtete Beziehung REPORTED_IN von COVID-19 nach Gelsenkirchen zu verwenden, um abzubilden, in welchen Städten eine Krankheit vorkommt. Durch einen Modellierungsfehler wurde stattdessen die Beziehung DISEASE_IN angelegt.

Abbildung 30

Modellierung Covid 19 Statistik von Stadt Gelsenkirchen-2

Im Gegensatz dazu zeigt Abbildung 31 die Modellierung für die Stadt Essen. Hier wurde ein einzelner StatType-Knoten *COVID-19 Essen* angelegt, der alle relevanten COVID-19-Statistiken als Eigenschaften enthält. Dadurch entfällt die Notwendigkeit mehrerer separater Knoten, und die statistischen Informationen werden kompakt und übersichtlich in einem einzigen Datenelement gespeichert.

Abbildung 31

Modellierung Covid 19 Statistik von Stadt Essen

Diese angepasste Modellierung in Essen bietet eine deutlich konsistentere und leichter analysierbare Struktur, da alle Zahlen zentral in einem Knoten abgelegt sind und eindeutig der Stadt zugeordnet werden können.

Zusammenfassend zeigt die Analyse der stark und schwach zusammenhängenden Komponenten, dass der Graph insgesamt stabil aufgebaut und gut strukturiert ist. Abgesehen von wenigen datenbedingten Besonderheiten wie der isolierten Conjunctivitis-(Pink Eye) -Gruppe oder den uneinheitlich modellierten COVID-19-Statistiken bleibt das Datenmodell weitgehend zusammenhängend. Vollständig vom übrigen Graphen getrennt ist nur die Conjunctivitis-(Pink Eye) -Komponente. Insgesamt liefert die Strukturanalyse damit eine verlässliche Grundlage für die folgenden Untersuchungen, in denen die Konnektivität des Graphen anhand des Durchmessers sowie der Gradzentralität genauer betrachtet wird.

4.2. Durchmesser-Analyse

Ziel dieser Analyse ist es, den Durchmesser des Graphen zu bestimmen. Dadurch lässt sich erkennen, wie gut die Daten miteinander verbunden sind und ob es Bereiche gibt, die nur über lange Ketten erreichbar sind. Die Analyse hilft außerdem, mögliche Modellierungsfehler oder fehlende Datenpunkte zu identifizieren und zeigt, an welchen Stellen das Modell erweitert oder verbessert werden könnte.

4.2.1. Methodik

Zur Ermittlung des Durchmessers wurde eine Cypher-Abfrage ausgeführt. Die Abfrage nutzt *gds.allShortestPaths.stream(),* um für alle möglichen Knotenpaare die kürzesten Pfade zu berechnen. Dieser Algorithmus berechnet für jeden Knotenpaaren in der Graphprojektion *myGraph* mögliche minimale kürzester Pfad (NEO4J docs, o. D.a). Die resultierenden Distanzen wurden anschließend absteigend sortiert. Es wird bis zu 10 Längste kürzester Pfad ausgegeben, das ist damit zu untersuchen, ob es mehrere Pfade mit gleiche Pfadlänge von andere Knotenpaaren existiert. Die verwendete Cypher-Abfrage ist in Abbildung 32 dargestellt.

Abbildung 32

Cypher-Abfrage zur Ermittlung des Durchmessers.

```
CALL gds.allShortestPaths.stream('myGraph')
YIELD sourceNodeId, targetNodeId, distance
RETURN sourceNodeId, targetNodeId, distance
ORDER BY distance DESC
LIMIT 10;
```

In der Abfrage werden die Werte sourceNodeId, targetNodeId und distance ausgegeben. Dabei handelt es sich um die internen Neo4j-KnotenIDs, die der Algorithmus für die Berechnung der kürzesten Wege verwendet.

Bei der Ausgabe bedeutet:

- **sourceNodeId** → interne ID des Startknotens
- **targetNodeId** → interne ID des Zielknotens
- **distance** → die berechnete Pfadlänge

Nachdem das Knotenpaar mit der maximalen Distanz bestimmt wurde, sollte auch der Pfad zwischen diesen beiden Knoten analysiert werden. Dafür wurde eine weitere Abfrage erstellt, die mit *shortestPath()* den tatsächlichen Weg, die beteiligten Knoten und die Kanten ausgibt. Es hilft zu identifizieren, welche Knotentyp und Kanten häufig in der Längster kürzester Pfad liegen. Die entsprechende Abfrage ist in Abbildung 33 dargestellt.

Abbildung 33

Cypher-Abfrage zur Visualisierung des längsten kürzesten Pfades.

```
MATCH (s),(t)
WHERE id(s) = 704 AND id(t) = 500
MATCH p = shortestPath((s)-[*]->(t))
RETURN
    p AS Pfad,
    nodes(p) AS Knoten,
    relationships(p) AS Kanten,
    length(p) AS Laenge;
```

Die Abfrage ermittelt den kürzesten Pfad zwischen den Knoten mit den internen IDs 704 und 500. Sie liefert anschließend den vollständigen Pfad zurück, einschließlich aller beteiligten Knoten und Kanten, sodass dieser visualisiert und weiter analysiert werden kann.

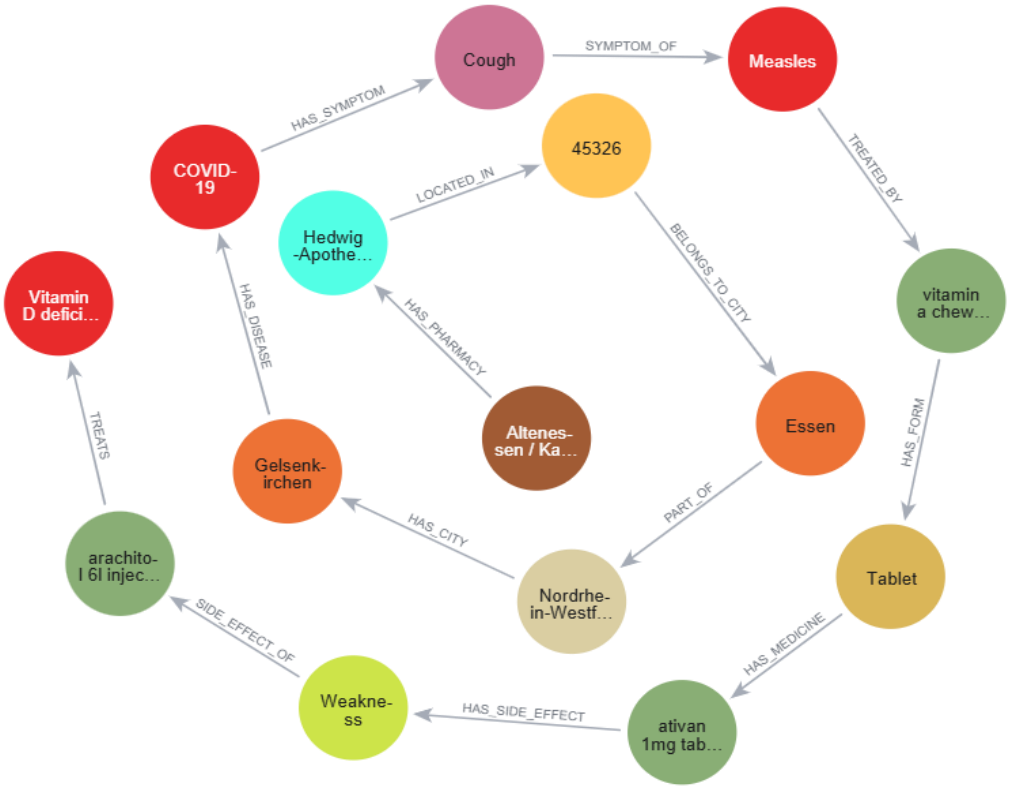
4.2.2. Ergebnisse

Die Analyse ergab einen Graphdurchmesser von 14. Dieser Maximalwert tritt bei vier verschiedenen längsten kürzesten Pfaden auf. In den meisten Fällen beginnen diese Pfade im Stadtbezirk *Altenessen / Karnap / Vogelheim* in Essen und enden bei einem Krankheits- oder Nebenwirkungsknoten. Einer der längsten kürzesten Pfade verläuft beispielsweise vom Stadtbezirksknoten *Altenessen / Karnap / Vogelheim* zur Krankheit *Vitamin-D-Mangel*.

Abbildung 34 zeigt den ermittelten längsten kürzesten Pfad, der in der Graphstruktur von Altenessen / Karnap / Vogelheim zur Krankheit Vitamin-D-Mangel führt.

Abbildung 34

Visualisierung des längsten kürzesten Pfades.



4.2.3. Interpretation der Ergebnisse

Die Ergebnisse zeigt, dass vor allem die medizinischen Entitäten wie Krankheiten, Symptome, Nebenwirkungen und Medikamente einen großen Einfluss auf die Pfadlänge des Durchmessers haben. Diese Knoten bilden häufig lineare Ketten, zum Beispiel Krankheit → Symptom → Krankheit → Medikament → Darreichungsform → Medikament und erhöhen dadurch die Gesamtdistanz.

Ein weiterer Faktor ist die unterschiedliche Modellierung der COVID-19-Statistiken. In Gelsenkirchen sind die statistischen Werte direkt über Beziehungseigenschaften mit dem COVID-19-Knoten verknüpft. In Essen hingegen wird ein zusätzlicher Statistik-Knoten verwendet. Dadurch entstehen längere Wege, die von Essen über Nordrhein-Westfalen und Gelsenkirchen bis in den medizinischen Bereich führen.

Die generelle Datenstruktur der Stadt Essen trägt zu längeren Pfaden bei. Während Gelsenkirchen durchgehende Verbindungen zwischen Postleitzahlen, Stadtbezirken, Stadtteilen und Einrichtungen besitzt, fehlen in Essen mehrere dieser Kanten. Dadurch müssen Pfade auf andere Bereiche des Graphen zurückgreifen und werden insgesamt länger.

4.2.4. Diskussion

Ein wichtiger Grund für den großen Durchmesser ist der medizinische Teil des Graphen. Die medizinischen Knoten haben nur wenige gemeinsame Merkmale und sind deshalb hauptsächlich in Form von langen, linearen Ketten miteinander verbunden. Einrichtungen wie Apotheken oder Arztpraxen besitzen dagegen geografische Gemeinsamkeiten, zum Beispiel Postleitzahlen oder Stadtteile. Dadurch entstehen dort viele kurze Verbindungen, die den Graphen insgesamt kompakter machen.

Im medizinischen Bereich sind solche Querverbindungen kaum vorhanden. Das liegt nicht an der Modellierung, sondern an den verfügbaren Daten, die nur sehr wenige gemeinsame Eigenschaften zwischen den medizinischen Entitäten enthalten.

Der Durchmesser könnte kleiner werden, wenn zusätzliche Verbindungen im Modell vorhanden wären. Dazu zählt beispielsweise eine VERFÜGBAR-Beziehung zwischen Medikamenten und Apotheken oder eine BIETET_AN-Beziehung zwischen Fachrichtungen und Krankenhäusern. Solche Verknüpfungen würden als Brücken zwischen dem medizinischen und dem geografischen Teil des Graphen wirken und damit die Pfadlängen deutlich verkürzen. Auch direkte REPORTED_IN-Beziehungen zwischen der Stadt Essen und bestimmten Krankheiten könnten zu kürzeren Wegen führen, da medizinische Informationen dann ohne Umwege erreichbar wären.

Die Umsetzung dieser zusätzlichen Verbindungen war im Rahmen dieser Arbeit jedoch nicht möglich. Für die Stadt Essen lagen deutlich weniger Daten vor als für Gelsenkirchen, was sich entsprechend in der Durchmesseranalyse zeigte. Da die Modellierung in einem Zeitraum von drei Monaten entstand und zunächst auf Gelsenkirchen fokussiert war, konnten entsprechende Erweiterungen für Essen nicht ergänzt werden. Dadurch bleiben potenzielle Verknüpfungen, die den Durchmesser verringert hätten, im aktuellen Modell unberücksichtigt.

4.3. Analyse der Grad-Zentralität

Das Ziel dieser Analyse ist es herauszufinden, wie gut die einzelnen Knoten im Graphen miteinander verbunden sind. Dazu werden der Eingangsgrad und der Ausgangsgrad berechnet. So können besonders wichtige Knoten erkannt werden, aber auch solche, die kaum Verbindungen haben. Die Ergebnisse helfen dabei zu verstehen, wie der Graph aufgebaut ist und wie vollständig die vorhandenen Daten das Modell abbilden.

4.3.1. Methodik

Für die Analyse der Gradzentralität wurden vier Cypher-Abfragen eingesetzt, um den höchsten und den niedrigsten Eingangsgrad sowie den höchsten und den niedrigsten Ausgangsgrad zu berechnen.

Eingangsgrad

Zur Berechnung des Eingangsgrads wurde eine Cypher-Abfrage verwendet, die alle Knoten erfasst und für jeden Knoten die Anzahl eingehender Beziehungen bestimmt. Dabei werden sämtliche gerichteten Kanten der Form *(m)* → *(n)* gezählt – also alle Verbindungen, die auf den jeweiligen Zielknoten *(n)* zeigen. Zusätzlich gibt die Abfrage den Knotennamen und seine zugehörigen Labels aus. Anschließend werden die Ergebnisse nach dem Eingangsgrad absteigend sortiert, um die fünf Knoten mit den meisten eingehenden Beziehungen zu ermitteln.

Das verwendete Abfrage ist in Abbildung 35 dargestellt.

Abbildung 35

Cypher-Abfrage zur Berechnung der fünf Knoten mit dem höchsten Eingangsgrad

```
MATCH (n)
RETURN
       n.name AS Name,
       labels(n) AS Knotentyp,
       COUNT { (m)-[]->(n) } AS Eingangsgrad
ORDER BY Eingangsgrad DESC
LIMIT 5;
```

Um auch Knoten mit einem sehr geringen Eingangsgrad zu identifizieren, wurde dieselbe Abfrage mit einer aufsteigenden Sortierung ausgeführt. Auf diese Weise lassen sich Knoten erkennen, die nur schwach oder gar nicht mit anderen Elementen des Graphen verbunden sind. Diese Ergebnisse sind besonders für die Beurteilung der Datenmodellqualität relevant, da fehlende eingehende Beziehungen auf isolierte oder unzureichend eingebundene Strukturen hinweisen können.

Auf eine Abbildung wurde hier verzichtet, da die Logik der Abfrage vollständig identisch ist.

Ausgangsgrad

Für die Bestimmung des Ausgangsgrads wurde eine weitere Cypher-Abfrage eingesetzt, die alle ausgehenden Beziehungen eines Knotens zählt. Dazu werden alle Kanten der Form *(n)* → *(m)* berücksichtigt, also alle Verbindungen, die vom jeweiligen Ausgangsknoten *(n)* zu anderen Knoten führen. Die Ergebnisse werden absteigend sortiert, sodass die fünf Knoten mit den meisten ausgehenden Beziehungen sichtbar werden. Das verwendete Abfragen ist in Abbildung 36 dargestellt.

Abbildung 36

Cypher-Abfrage zur Berechnung der fünf Knoten mit dem höchsten Ausgangsgrad

```
MATCH (n)
RETURN
       n.name AS Name,
       labels(n) AS Knotentyp,
       COUNT { (n)-[]->(m) } AS Ausgangsgrad
ORDER BY Ausgangsgrad DESC
LIMIT 5;
```

Ergänzend wurde der Ausgangsgrad auch in invertierter Sortierung analysiert, um Knoten mit besonders wenigen ausgehenden Beziehungen zu erkennen. Diese Knoten initiieren kaum Verbindungen und können daher Hinweise auf isolierte oder funktional eingeschränkte Bereiche im Datenmodell liefern.

Auf eine Abbildung wurde hier auch verzichtet.

Durch die Kombination aller vier Abfragen entsteht ein umfassendes Bild der strukturellen Einbindung der Knoten. Sie zeigt sowohl besonders stark vernetzte als auch schwach angebundene Elemente und erlaubt damit eine fundierte Bewertung der Konnektivität des gesamten Graphen.

4.3.2. Ergebnisse

Eingangsgrad

Tabelle 1 zeigt das Ergebnis der fünf höchsten Eingangsgrade. Im Anschluss werden im Text auch die Knoten mit den niedrigsten Eingangsgraden beschrieben.

Tabelle 1

Knoten mit den höchsten Eingangsgraden

Name	Knotentyp	Eingangsgrad
45879	PostalCode	100
45894	PostalCode	99
Hausärztliche Versorgung	Specialty	96
Nausea	SideEffect	73
Allgemeinmedizin	Specialty	62

Die Analyse der **niedrigsten Eingangsgrade** zeigt, dass mehrere Knoten lediglich eine eingehende Beziehung aufweisen. Dazu gehören die Fachrichtungen *Unfallchirurgie*, *Psychosomatische Medizin* und *Medizinethik* sowie die Nebenwirkungen *Blisters* und *Tingling*. Da alle betroffenen Knoten denselben minimalen Wert besitzen, wurde auf eine tabellarische Darstellung verzichtet.

Ausgangsgrad

Tabelle 2 zeigt die fünf höchsten Ausgangsgrade. Auch hier werden im Text danach die Knoten mit den niedrigsten Ausgangsgraden erläutert.

Tabelle 2

Knoten mit den höchsten Ausgangsgraden

Name	Knotentyp	Ausgangsgrad
45894	PostalCode	97
45879	PostalCode	97
Nausea	SideEffect	73
Mitte	District	66
Tablet	MedicineForm	61

Die Analyse der **niedrigsten Ausgangsgrade** zeigt, dass mehrere Fachrichtungen keinerlei ausgehende Beziehungen besitzen. Dazu gehören *Augenheilkunde*, *Psychosomatische Medizin*, *Hausärztliche Versorgung*, *Allgemeinmedizin* und *Geschlechtskrankheiten*, die jeweils einen Ausgangsgrad von 0 aufweisen. Da alle betroffenen Knoten denselben minimalen Wert besitzen, wurde auch hier auf eine tabellarische Darstellung verzichtet

4.3.3. Interpretation der Ergebnisse

Die Ergebnisse zeigt, dass die Postleitzahlknoten *45879* und *45894* die höchsten Eingangs- und Ausgangsgrade besitzen. Postleitzahlen spielen im Modell eine zentrale Rolle, da über sie viele Einrichtungen, Arztpraxen und Stadtbezirke miteinander verknüpft sind. Dadurch nehmen sie im geografischen Teil des Graphen eine besonders wichtige strukturelle Rolle ein.

Auch im medizinischen Bereich werden klare Zusammenhänge sichtbar. Die Fachrichtungen *Hausärztliche Versorgung* und *Allgemeinmedizin* haben hohe Eingangsgrade, werden aber beim Ausgangsgrad nicht sichtbar. Das bedeutet, dass viele medizinische Knoten zu diesen Fachrichtungen zeigen, sie selbst aber keine weiteren Verbindungen starten. Fachrichtungen dienen damit im Modell vor allem als Zielknoten.

Darüber hinaus fallen die Nebenwirkung Knoten *Nausea* und Darreichungsform *Tablet* durch hohe Gradwerte auf. Dies ist inhaltlich gut nachvollziehbar: Nausea ist eine häufige Nebenwirkung vieler Medikamente, und Tabletten stellen eine weit verbreitete Darreichungsform dar. Beide Knoten sind daher mit zahlreichen Medikamenten verbunden.

Zusammenfassend zeigt die Analyse eine klare Struktur:

- Im **geografischen Teil** dominieren Postleitzahlen als zentrale Knoten.
- Im **medizinischen Teil** übernehmen Nebenwirkungen wie *Nausea* und Darreichungsformen wie *Tablet* zentrale Rollen.
- **Fachrichtungen** fungieren überwiegend als Zielknoten und besitzen eine eher passive Rolle im Netzwerk.

4.3.4. Diskussion

Die Analyse zeigen, dass die Struktur des Graphen nicht nur von der Menge der vorhandenen Daten abhängt, sondern auch davon, wie die Stadt aufgebaut ist. In Gelsenkirchen gibt es viele gut strukturierte Daten. Außerdem liegen die Postleitzahlen *45879* und *45894* in zentralen Bereichen der Stadt. Dort befinden sich besonders viele medizinische Einrichtungen, Apotheken und Haltestellen, die eng miteinander verbunden sind. Deshalb spielen diese Postleitzahlen im Graphen eine deutlich wichtigere Rolle.

Für Essen wurden bisher deutlich weniger Daten integriert, weshalb die dortigen Knoten weniger stark vernetzt erscheinen. Es ist jedoch anzunehmen, dass Postleitzahlen im Essener Stadtzentrum ähnliche zentrale Rollen einnehmen könnten wie in Gelsenkirchen, sofern mehr Daten aus dieser Region eingebunden werden.

Ein weiterer Punkt betrifft die Fachrichtungsknoten. Sie besitzen viele eingehende, aber keine ausgehenden Beziehungen. Dies liegt daran, dass im aktuellen Datenbestand Informationen fehlen, die zeigen würden, welche Krankheiten zu welcher Fachrichtung gehören oder welche Krankenhäuser bestimmte Fachbereiche anbieten. Mit einer umfassenderen Datenbasis könnten Fachrichtungen deutlich aktiver vernetzt sein.

Insgesamt zeigt die Analyse der Gradzentralität, dass die Graphmodellierung zuverlässig funktioniert und bereits realistische Analysen ermöglicht. Die Ergebnisse könnten zwar durch zusätzliche Daten, besonders im medizinischen Bereich und in weiteren Stadtteilen, noch präziser werden. Trotzdem bietet das aktuelle Modell bereits eine gute Grundlage. Darauf aufbauend können nun praktische Fragestellungen untersucht werden, zum Beispiel die Erkennung von Krankheiten anhand von Symptomen.

4.4. Krankheitserkennung anhand von Symptomen

Ziel dieser Analyse ist es zu prüfen, wie gut ein Graphmodell dabei helfen kann, mögliche Krankheiten aus einer gegebenen Symptomliste zu bestimmen. Dabei soll untersucht werden, welche Krankheiten die meisten Übereinstimmungen mit den eingegebenen Symptomen haben und ob der Graph diese Verbindungen zuverlässig erkennt. Außerdem soll bewertet werden, ob ein solches System eine erste Orientierung bei der Diagnose unterstützen kann.

4.4.1. Methodik

Für den Test wurde eine Liste von elf Symptomen verwendet, darunter *Abdominal pain, Fever* und *Sore throat*. Die Abfrage durchsucht den Graphen nach Krankheiten, die mit diesen Symptomen verbunden sind, und zählt, wie viele Treffer es pro Krankheit gibt. Dadurch lässt sich erkennen, welche Diagnose anhand der vorhandenen Daten am wahrscheinlichsten ist. Die verwendete Abfrage ist in Abbildung 37 dargestellt.

Abbildung 37

Cypher Abfrage zur Krankheitsdiagnose.

```
WITH [
    "Abdominal pain", "Body aches", "Fever", "Headache",
    "Loss of appetite", "Nausea", "Rash", "Red and swollen tonsils",
    "Sore throat", "Swollen lymph nodes", "White patches on tonsils"
] AS symptomList
MATCH (s:Symptom)-[:SYMPTOM_OF]->(d:Disease)
WHERE s.name IN symptomList
WITH d, collect(s.name) AS matchedSymptoms,
     count(s) AS matchedCount,
     size(symptomList) AS totalSymptoms
RETURN
    d.name AS Krankheit,
    matchedSymptoms AS Gefundene_Symptome,
    matchedCount AS Trefferanzahl
ORDER BY matchedCount DESC
LIMIT 3;
```

Die Abfrage startet mit einer Liste von Symptomen, die als Grundlage für den Test dient. Anschließend werden alle Symptom-Knoten gesucht, die in dieser Liste vorkommen und über die Beziehung **SYMPTOM_OF** mit einer Krankheit verbunden sind. Für jede gefundene Krankheit werden dann drei wichtige Berechnungen durchgeführt:

- **collect(s.name)** sammelt alle passenden Symptom-Namen in einer Liste. Dadurch lässt sich später nachvollziehen, welche Symptome zu einer bestimmten Krankheit gefunden wurden.
- **count(s)** zählt, wie viele dieser Symptome bei einer Krankheit tatsächlich übereinstimmen. Das entspricht der Trefferzahl.
- **size(symptomList)** bestimmt, wie viele Symptome insgesamt in die Liste aufgenommen wurden. Dadurch wird klar, aus wie vielen möglichen Symptomen die Treffer stammen.

Im letzten Schritt gibt die Abfrage zu jeder Krankheit den Namen, die gefundenen Symptome und die Anzahl der Treffer zurück. Die Ergebnisse werden anschließend nach der höchsten Trefferzahl sortiert. Dadurch erscheinen jene Krankheiten zuerst, die am stärksten mit den eingegebenen Symptomen übereinstimmen und somit im Kontext der Daten als am wahrscheinlichsten gelten.

4.4.2. Ergebnisse

Tabelle 3 zeigt die drei Krankheiten mit den meisten passenden Symptomen:

Tabelle 3

Ergebnis der Krankheitsdiagnose.

Krankheit	Gefundene_Symptome	Trefferanzahl
Strep Throat	Abdominal pain, Body aches, Fever, Headache, Loss of appetite, Nausea, Rash, Red and swollen tonsils, Sore throat, Swollen lymph nodes, White patches on tonsils	11
Roseola	Abdominal pain, Body aches, Headache, Loss of appetite, Rash, Sore throat, Swollen lymph nodes]	7
Norovirus Infection	Abdominal pain, Body aches, Fever, Headache, Loss of appetite, Nausea	6

Die Auswertung macht deutlich, dass die Krankheit *Strep Throat* alle eingegebenen Symptome enthält und somit die größte Übereinstimmung besitzt. Die anderen beiden Krankheiten weisen deutlich weniger Symptome aus der Liste auf.

4.4.3. Interpretation der Ergebnisse

Die Ergebnisse zeigen, dass der Graph zuverlässig die Krankheiten findet, die am besten zu den eingegebenen Symptomen passen. *Strep Throat* erscheint an erster Stelle, weil alle elf eingegebenen Symptome in der Datenbank mit dieser Krankheit verbunden sind. Das bedeutet, dass diese Diagnose auf Basis der vorliegenden Daten am wahrscheinlichsten ist.

Krankheiten wie *Roseola* oder *Norovirus Infection* haben deutlich weniger Übereinstimmungen. Dadurch werden sie niedriger eingestuft. Diese Analyse verdeutlicht, dass der Graph in der Lage ist, typische Symptom-Krankheits-Muster zu erkennen und automatisch zu priorisieren.

Insgesamt zeigt die Auswertung, dass ein Graphmodell Symptome strukturiert mit Krankheiten verknüpfen kann und damit eine automatische Unterstützung bei der ersten Einschätzung möglicher Diagnosen bietet.

4.4.4. Diskussion

Die Analyse zeigt, dass das Graphmodell gut funktioniert, um passende Krankheiten zu einer gegebenen Symptomliste zu finden. Das Modell erkennt typische Verbindungen zwischen Symptomen und Krankheiten und ordnet die Ergebnisse nach der Anzahl der Übereinstimmungen. Dadurch kann es eine hilfreiche Unterstützung bei einer ersten Einschätzung möglicher Diagnosen sein.

Trotz dieser positiven Ergebnisse gibt es einige Punkte, die die Aussagekraft der Analyse einschränken können.

Erstens treten viele Symptome wie Fieber, Kopfschmerzen oder Übelkeit bei sehr vielen Krankheiten auf. Solche unspezifischen Symptome führen dazu, dass mehrere Krankheiten ähnliche Trefferwerte erhalten können. Dadurch wird die Diagnose allein auf Basis der Symptome ungenauer.

Zweitens fehlen im aktuellen Datenbestand viele wichtige Zusatzinformationen. Für eine medizinisch zuverlässige Bewertung wären weitere Daten notwendig, zum Beispiel Alter, Vorerkrankungen, Risikofaktoren, Laborwerte oder zeitliche Entwicklung der Symptome. Ohne diese Informationen kann die Analyse nur eine grobe Einschätzung liefern, aber keine belastbare Diagnose.

Drittens ist die Qualität der Ergebnisse stark davon abhängig, wie vollständig die Datenbank modelliert wurde. Wenn Krankheiten, Symptome oder Medikamentenbeziehungen fehlen, beeinflusst das direkt die Trefferanzahl und damit auch die errechnete Wahrscheinlichkeit.

Insgesamt zeigt die Analyse, dass ein Graphmodell geeignet ist, Symptome strukturiert mit Krankheiten zu verknüpfen und mögliche Diagnosen sinnvoll vorzuschlagen. Es kann dabei unterstützen, erste Hinweise zu liefern und Erkrankungen nach ihrer Wahrscheinlichkeit zu sortieren. Gleichzeitig bleibt die Genauigkeit der Ergebnisse begrenzt, da unspezifische Symptome, fehlende Zusatzinformationen und ein unvollständiger Datenbestand die Aussagekraft einschränken. Auf dieser Grundlage lässt sich nun eine weitere praktische Anwendung analysieren, nämlich der Zusammenhang zwischen Einwohnerzahl und Apothekenverteilung.

4.5. Einwohnerzahl und Apothekenverteilung

Das Ziel dieser Analyse ist es zu untersuchen, wie viele Einwohner in Essen und Gelsenkirchen auf eine Apotheke kommen. So soll sichtbar werden, wie unterschiedlich die beiden Städte mit Apotheken versorgt sind und ob das Graphmodell solche Vergleiche gut darstellen kann.

4.5.1. Methodik

Für das Analyse wurden Cypher-Abfragen erstellt, die sowohl die jeweilige Einwohnerzahl als auch die Anzahl der vorhandenen Apotheken aus dem Graphen ermitteln.

Die verwendeten Abfragen sind in Abbildung 38 für Essen und in Abbildung 39 für Gelsenkirchen dargestellt.

Abbildung 38

Cypher-Abfrage zur Berechnung der Einwohner-pro-Apotheke-Relation in Essen

```
MATCH (s:StatType {name: "Einwohner Essen"})
WITH s.value AS einwohner
MATCH (p:Pharmacy)-[:LOCATED_IN]->(d:District {city: "Essen"})
RETURN
    einwohner AS Einwohner,
    count(p) AS Apotheken,
    round(einwohner * 1.0 / count(p), 2) AS Einwohner_pro_Apotheke;
```

Für die Analyse in Essen wird zunächst die Einwohnerzahl der Stadt Essen aus dem entsprechenden *StatType*-Knoten ausgelesen. Dieser Knoten enthält statistische Informationen, die als Grundlage für die Berechnung dienen. Anschließend werden alle Apotheken ermittelt, die über eine **LOCATED_IN**-Beziehung einem Stadtteil von Essen zugeordnet sind. Dadurch wird sichergestellt, dass nur Apotheken berücksichtigt werden, die tatsächlich innerhalb der Stadtgrenzen liegen.

Im nächsten Schritt bestimmt die Funktion **count(p)** die Gesamtzahl der gefundenen Apotheken. Durch die Division der Einwohnerzahl durch diese Apothekenzahl wird berechnet, wie viele Einwohner durchschnittlich auf eine Apotheke entfallen. Das Ergebnis wird mit **round(..., 2)** auf zwei Dezimalstellen gerundet, um eine gut vergleichbare Kennzahl zu erhalten.

Abbildung 39

Cypher-Abfrage zur Berechnung der Einwohner-pro-Apotheke-Relation in Gelsenkirchen.

```
MATCH (s:StatType {name: "Einwohner Gelsenkirchen"})
WITH s.value AS einwohner
MATCH (p:Pharmacy {city: "Gelsenkirchen"})
RETURN
    einwohner AS Einwohner,
    count(p) AS AnzahlApotheken,
    ROUND(einwohner * 1.0 / count(p),2) AS Einwohner_pro_Apotheke;
```

Für die Stadt Gelsenkirchen wurde eine leicht angepasste Abfrage verwendet. Da Apotheken in diesem Teil des Datenbestands direkt über das Attribut **city = "Gelsenkirchen"** identifiziert werden können, entfällt hier der Zwischenschritt über einen District-Knoten. Die Berechnung folgt jedoch dem gleichen Prinzip: Zunächst wird die Einwohnerzahl aus dem passenden *StatType*-Knoten ausgelesen, anschließend die Anzahl der Apotheken gezählt und daraus der Durchschnitt der Einwohner pro Apotheke berechnet und gerundet.

4.5.2. Ergebnisse

Die Tabellen 4 und 5 zeigen die berechneten Werte.

Tabelle 4

Anzahl der Einwohner und Apotheken in Essen.

Einwohner	AnzahlApotheken	Einwohner_pro_Apotheke
579432	110	5267.56

Die Auswertung zeigt, dass Essen über 579.432 Einwohner und 110 Apotheken verfügt. Daraus ergibt sich ein Durchschnitt von 5.267,56 Einwohnern pro Apotheke.

Tabelle 5

Anzahl der Einwohner und Apotheken in Gelsenkirchen.

Einwohner	AnzahlApotheken	Einwohner_pro_Apotheke
260126	34	7650.76

Für Gelsenkirchen wurden 260.126 Einwohner und 34 Apotheken identifiziert. Dies entspricht einem Durchschnitt von 7.650,76 Einwohnern pro Apotheke.

Im direkten Vergleich zeigt sich, dass Essen eine höhere Apothekendichte besitzt, da dort im Durchschnitt weniger Einwohner auf eine einzelne Apotheke entfallen.

4.5.3. Interpretation der Ergebnisse

Die Ergebnisse zeigen, dass die Apothekendichte in Essen höher ist als in Gelsenkirchen. In Essen kommen durchschnittlich etwa 5.268 Einwohner auf eine Apotheke, während es in Gelsenkirchen rund 7.651 Einwohner pro Apotheke sind.

Damit hat Essen im Verhältnis zur Einwohnerzahl mehr Apotheken. Das bedeutet, dass die Menschen in Essen einen besseren und schnelleren Zugang zu einer Apotheke haben als in Gelsenkirchen.

Der Vergleich macht deutlich, dass das Graphmodell solche Unterschiede gut sichtbar machen kann und damit hilft, regionale Versorgungsstrukturen zu verstehen.

4.5.4. Diskussion

Die Analyse zeigt, dass Essen und Gelsenkirchen unterschiedlich versorgt sind. Solche Unterschiede sind wichtig für die Planung, weil sie Hinweise darauf geben, wo zusätzliche Apotheken, Pflegeeinrichtungen oder andere medizinische Angebote sinnvoll sein könnten.

Grundsätzlich lässt sich das Verfahren auch auf andere Einrichtungen übertragen, etwa Krankenhäuser, Notdienste oder Pflegeheime. Die Berechnung liefert dabei einen ersten Überblick darüber, wie dicht die medizinische Versorgung in einer Region ist.

Für eine bessere Analyse wären demografische Daten, wie Altersgruppen, hilfreich, da sie einen direkten Einfluss auf den Bedarf bestimmter Einrichtungen haben etwa Pflegeheime oder medizinische Dienste.

Darüber hinaus zeigt die Analyse, dass Graphdatenbanken für solche Untersuchungen gut geeignet sind. Im Gesundheitswesen sind viele Informationen miteinander verknüpft, zum Beispiel zwischen medizinischen Einrichtungen, Statistiken und räumliche Daten. Ein Graph kann diese Beziehungen übersichtlich darstellen und flexibel auswerten. Je mehr relevante Verbindungen im Graph enthalten sind, desto genauer lassen sich regionale Versorgungsstrukturen untersuchen.

5. Diskussion

Die Analysen zeigen, dass das entwickelte Graphmodell insgesamt gut funktioniert. Die medizinischen und geografischen Daten wurden sinnvoll miteinander verbunden, sodass ein großer zusammenhängender Graph entstanden ist. Die Ergebnisse zu den stark und schwach zusammenhängenden Komponenten zeigen, dass der Graph größtenteils gut vernetzt ist und nur einen vollständig isolierten Bereich enthält. Auch die weiteren Analysen wie der Durchmesser, die Gradzentralität, die Krankheitserkennung und die Untersuchung der Apothekendichte machen deutlich, dass das Modell logische und realistische Ergebnisse liefert.

Ein Vergleich der Analysenmethoden zeigt, dass jede Analyse einen eigenen Blick auf den Graphen gibt. Die Analyse der zusammenhängenden Komponenten zeigt, wie gut der Graph insgesamt verbunden ist. Die Untersuchung des Durchmessers macht sichtbar, an welchen Stellen besonders lange Pfade auftreten und wo potenziell Verbindungen fehlen. Die Berechnung der Gradzentralität zeigt, welche Knoten eine zentrale Rolle einnehmen, beispielsweise einzelne Postleitzahlen oder häufig vorkommende Nebenwirkungen. Die praxisorientierten Analysen, wie die Krankheitserkennung und der Vergleich der Apothekendichte, verdeutlichen zudem die Anwendbarkeit des Modells für reale Fragestellungen. Das zeigt, dass das Graphmodell ist klar strukturiert und verknüpft die unterschiedlichen Datenbereiche gut miteinander.

Gleichzeitig wurden einige Schwächen sichtbar. Vor allem im medizinischen Bereich gibt es nur wenige Verbindungen zwischen den einzelnen Knoten. Dadurch entstehen lange Pfade, und die Konnektivität ist geringer. Beispielweise die Krankheitsgruppe *Conjunctivitis (Pink Eye)* ist isoliert, weil passende Daten fehlen. Diese Schwächen liegen vor allem an den begrenzten verfügbaren Daten. Es beeinflusst die Ergebnisse der Analysen. Für die Krankheit *Conjunctivitis (Pink Eye)* kann keine Medikamente Erkennung gemacht werden, weil die Medikamente Daten für *Conjunctivitis (Pink Eye)* fehlen.

Ein weiterer Punkt betrifft die Datenqualität und Vollständigkeit. Für die Stadt Gelsenkirchen standen viele strukturierte Daten zur Verfügung, während für Essen nur wenige Informationen vorhanden waren. Dadurch war eine vollständige Modellierung für Essen nicht möglich. Infolgedessen ist der resultierende Graph ungleich verteilt.

Eine Schwäche der Modellierung besteht darin, dass die Daten für Gelsenkirchen und Essen nicht einheitlich strukturiert waren. Dadurch fiel die Modellierung in beiden Städten unterschiedlich aus, was sich insbesondere in der Analyse der Zusammenhangskomponenten und des Durchmessers bemerkbar machte. Für einen Vergleich der COVID-19-Statistiken hätten die entsprechenden Abfragen sogar vollständig neu formuliert werden müssen.

Die Modellierung in Essen wurde später begonnen, weil am Anfang nur Gelsenkirchen eingeplant war. Das hat die Analyse eingeschränkt. Zum Beispiel konnte der wichtige Vergleich zur Krankenhausdichte nicht durchgeführt werden, weil in Essen keine Krankenhäuser modelliert wurden. Das zeigt, dass die allgemeinen Ergebnisse stark von der Qualität und Menge der Daten sowie von der Modellierung der Datenbank abhängen.

Trotz der Einschränkungen zeigt der Graph insgesamt eine gute Funktion. Für noch genauere Ergebnisse wären jedoch vollständigere, konsistentere und einheitlich strukturierte Daten und klarere Modellierung notwendig.

6. Schlussfolgerung

Dieses Kapitel fasst die wichtigsten Ergebnisse der Arbeit zusammen und gibt einen Ausblick auf mögliche zukünftige Erweiterungen des entwickelten Graphmodells.

6.1. Fazit

Die Analysen zeigen, dass sich medizinische und räumliche Daten gut in einer Graphdatenbank abbilden lassen. Durch eine klare Struktur und eindeutige Beziehungstypen konnten beide Bereiche logisch miteinander verknüpft werden. Das Modell ist klar strukturiert und erlaubt verschiedene Analysen ohne große Anpassungen.

Beim Aufbau der Datenbank traten jedoch einige Schwierigkeiten auf. Dazu gehörten fehlende oder unvollständige Daten, unterschiedliche Formate sowie wenige Daten im medizinischen Bereich mit Zusammenhang. Diese Faktoren führten zu längeren Pfaden und einzelnen isolierten Knoten. Die Herausforderungen lagen damit weniger im Modell selbst, sondern vor allem in der Datenqualität und Verfügbarkeit.

Trotz dieser Einschränkungen zeigt die Arbeit, dass eine Graphdatenbank für praktische Fragestellungen im Gesundheitswesen gut geeignet ist. Anwendungen wie das Erkennen möglicher Krankheiten anhand von Symptomen oder der Vergleich von Versorgungsstrukturen lassen sich direkt umsetzen. Damit bietet das Modell eine gute Grundlage für weitere Untersuchungen und für zukünftige Erweiterungen.

6.2. Ausblick

Das entwickelte Graphmodell bietet gute Möglichkeiten für zukünftige Arbeiten in der Versorgungsforschung. In Zukunft kann das Projekt durch zusätzliche und vielfältigere Daten weiter ausgebaut werden. Dazu gehört die Integration verschiedener Datenquellen sowie die Erweiterung auf weitere Städte oder ganze Regionen, um räumliche Vergleiche besser durchführen zu können. Auch die Einbindung von Echtzeitdaten wäre sinnvoll, zum Beispiel Informationen darüber, ob Krankenhäuser oder Arztpraxen neue Patientinnen und Patienten aufnehmen oder ob bestimmte Medikamente aktuell in Apotheken verfügbar sind.

Darüber hinaus kann die Datenstruktur erweitert werden. Fehlende Beziehungstypen wie **VERFÜGBAR_IN** (zwischen Medikamenten und Apotheken), **BIETET_AN** (zwischen Krankenhäusern und Fachrichtungen) oder **BEHANDELT** (zwischen Ärzten und Krankheiten) würden den Graphen realistischer machen und neue Analyseformen ermöglichen.

Das Modell bietet auch Potenzial für praktische Anwendungen. Dazu zählen **benutzerfreundliche Oberflächen** für medizinisches Personal oder **mobile Apps** für die Bevölkerung. Beispiele wären ein Apothekenfinder mit aktueller Medikamentenverfügbarkeit oder eine Anwendung, die das nächstgelegene Krankenhaus mit bestimmter Fachrichtung anzeigt. Ein weiteres mögliches Einsatzfeld ist ein **automatisches Warnsystem**, das steigende Krankheitszahlen in benachbarten Städten oder Regionen erkennt.

Diese möglichen Erweiterungen zeigen, dass das Modell eine gute Grundlage für weitere Forschung und praktische Lösungen im Gesundheitswesen darstellt und zukünftig flexibel ausgebaut werden kann.

Insgesamt zeigt die Arbeit, dass Graphdatenbanken eine hilfreiche Grundlage für medizinische und geografische Analysen bieten und in zukünftigen Forschungs- und Praxisprojekten wertvolle Beiträge leisten können.

Literaturverzeichnis

Angles, R. & Gutierrez, C. (2008). Survey of graph database models. *ACM Computing Surveys*, *40*(1), 1–39. https://doi.org/10.1145/1322432.1322433

Ashariana, A. (2021). *Infectious disease dataset*. https://zenodo.org/records/13338116 https://doi.org/10.5281/ZENODO.13338115

Berghammer, R. (2024). *Mathematik für die Informatik* (5. Aufl.). Springer Fachmedien Wiesbaden. https://doi.org/10.1007/978-3-658-44149-4

Bondy, J. A. & Murty, U. S. R. (2008). *Graph Theory* (Bd. 244). Springer London. https://doi.org/10.1007/978-1-84628-970-5

Georgiadis, L., Italiano, G. F. & Parotsidis, N. (2020). Strong Connectivity in Directed Graphs under Failures, with Applications. *SIAM Journal on Computing*, *49*(5), 865–926. https://doi.org/10.1137/19M1258530

Heun, V. (2003). *Grundlegende Algorithmen*. 2. https://link.springer.com/chapter/10.1007/978-3-322-80323-8_5

Needham, M. & Hodler, A. E. (2019). *Graph algorithms: Practical examples in Apache Spark and Neo4j* (First edition). O'Reilly Media.

Neo4J. (o. D.). *What is Neo4j?* Zugriff am 2. Oktober 2025, verfügbar unter https://neo4j.com/docs/getting-started/whats-neo4j/

NEO4J docs. (o. D.a). *Algorithms procedures*. https://neo4j.com/docs/graph-data-science-client/current/api/algorithms/

NEO4J docs. (o. D.b). *Cypher projection*. Zugriff am 2. Dezember 2025, verfügbar unter https://neo4j.com/docs/graph-data-science/current/management-ops/graph-creation/graph-project-cypher-projection/

NEO4J docs. (o. D.c). *MATCH*. Zugriff am 3. Oktober 2025, verfügbar unter https://neo4j.com/docs/cypher-manual/current/clauses/match/

NEO4J docs. (o. D.d). *Strongly Connected Components*. Zugriff am 4. November 2025, verfügbar unter https://neo4j.com/docs/graph-data-science/current/algorithms/strongly-connected-components/

NEO4J docs. (o. D.e). *Weakly Connected Components*. https://neo4j.com/docs/graph-data-science/current/algorithms/wcc/

OpenStreetMap Foundation. (2025). *OpenStreetMap – Copyright and License*. https://www.openstreetmap.org/copyright

Robinson, I., Webber, J. & Eifrem, E. (2015). *Graph databases* (Second edition). O'Reilly.

Shudhanshu, S., Vivek, T. & Vishal, T. (2022). *250k Medicines Usage, Side Effects and Substitutes*. https://www.kaggle.com/datasets/shudhanshusingh/250k-medicines-usage-side-effects-and-substitutes

Siekmann, M. (2025a). *Corona-Zahlen für Essen*. https://www.corona-in-zahlen.de/landkreise/sk%20essen/

Siekmann, M. (2025b). *Corona-Zahlen für Gelsenkirchen*. https://www.corona-in-zahlen.de/landkreise/sk%20gelsenkirchen/

Stadt Essen. (2025). *Digital Open Data*. https://opendata.essen.de/search/type/dataset

Stadt Gelsenkirchen. (2025). *Open Data Portal Stadt Gelsenkirchen*. https://gelsenkirchen.opendata.ruhr/dataset/

Teschl, G. & Teschl, S. (2013). *Mathematik für Informatiker*. Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-642-37972-7